

Scalable and Fine-Tuned Privacy Pass from Group Verifiable Random Functions

Dennis Faut*

University of Luxembourg & KASTEL SRL
Karlsruhe, Germany
dennis.faut@kit.edu

Julia Hesse†

IBM Research Europe
Zurich, Switzerland
jhs@zurich.ibm.com

Lisa Kohl‡

Centrum Wiskunde & Informatica (CWI)
Amsterdam, Netherlands
lisa.kohl@cw.nl

Andy Rupp*

University of Luxembourg & KASTEL SRL
Belval, Luxembourg
andy.rupp@uni.lu

Abstract—Anonymous token schemes are cryptographic protocols for limiting the access to online resources to credible users. The resource provider issues a set of access tokens to the credible user that they can later redeem anonymously, i.e., without the provider being able to link their redemptions. When combined with credibility tests such as CAPTCHAs, anonymous token schemes can significantly increase user experience and provider security, without exposing user access patterns to providers.

Current anonymous token schemes such as the Privacy Pass protocol by Davidson et al. rely on *oblivious pseudorandom functions* (OPRFs), which let server and user jointly compute randomly looking access tokens. For those protocols, token issuing costs are linear in the number of requested tokens.

In this work, we propose a new approach for building anonymous token schemes. Instead of relying on two-party computation to realize a privacy-preserving pseudorandom function evaluation, we propose to offload token generation to the user by using *group verifiable random functions* (GVRFs). GVRFs are a new cryptographic primitive that allow users to produce verifiable pseudorandomness. Opposed to standard VRFs, verification is *anonymous within the group* of credible users. We give a construction of group VRFs from the Dodis-Yampolskiy VRF and Equivalence-Class Signatures, based on pairings and a new Diffie-Hellman inversion assumption that we analyze in the Generic Group Model. Our construction enjoys compact public keys and proofs, while evaluation and verification costs are only slightly increased compared to the Dodis-Yampolskiy VRF.

By deploying a group VRF instead of a OPRF, we obtain an anonymous token scheme where communication as well as server-side computation during the issuing phase is constant and independent of the number of tokens a user requests. Moreover, by means of our new concept of

updatable token policies, the number of unspent tokens in circulation can *retrospectively* (i.e., even after the credibility check) be decreased or increased in order to react to the current or expected network situation. Our tokens are further countable and publicly verifiable. This comes at the cost of higher computational efforts for token redemption and verification as well as somewhat weaker unlinkability guarantees compared to Privacy Pass.

Index Terms—Verifiable random functions, anonymous token schemes, Privacy Pass, pairing-based cryptography

1. Introduction

CAPTCHAs are proofs of human work that protect internet resources from bot access and DoS attacks. Users are asked to solve a visual riddle, such as reading distorted letters or recognizing items in photographs, whenever a content provider deems their request suspicious. Solving the riddle reveals nothing about the identity of the user except that, most likely, there is a human behind the request. In particular, providers cannot *link* CAPTCHA actions, preventing them from tracking users on the internet. On the negative side, CAPTCHAs constitute annoying obstacles for users and can prevent them from accessing contents [22].

In 2018, Davidson et al. [15] proposed Privacy Pass, a way to significantly decrease the number of CAPTCHAs for users to solve. In a nutshell, the protocol works as follows. First, the server (i.e., content provider) is set up to hold a secret key K . Whenever a user proves that she is human via a CAPTCHA, the user's browser and server engage in an oblivious evaluation of $w := \text{PRF}_K(t)$, where t is chosen at random by the browser. Such a protocol is called an oblivious pseudo-random function (OPRF), and it reveals nothing to the user beyond w , and nothing at all to the server. We call the pair (t, w) an *anonymous access token*, and the browser can use it next time the server doubts the user's credibility: instead of challenging the user with a CAPTCHA, the browser

* Supported by funding from the topic Engineering Secure Systems of the Helmholtz Association (HGF) and by KASTEL Security Research Labs.

† Supported by the Swiss National Science Foundation (SNSF) under the AMBIZIONE grant "Cryptographic Protocols for Human Authentication and the IoT".

‡ Supported by the NWO Talent Programme Veni (VI.Veni.222.348) and the NWO Gravitation Project QSC.

sends $(t, w)^1$. The server is convinced that the user has previously solved a CAPTCHA, as there is no other way to compute w such that $w = \text{PRF}_K(t)$. Crucially, the spending of these tokens is *anonymous* and even *unlinkable*, meaning that the server has no means to determine which tokens have been generated by the same user. This property is ensured by the obliviousness of the PRF evaluation, where the server does not learn anything about the input t and the output w of the user. To save even more CAPTCHA work, [15] suggests that the server can agree to issue multiple tokens per completed CAPTCHA. Of course, this not only speeds up accesses but also reduces the number of CAPTCHAs that an attacker has to solve when gathering Privacy Pass tokens to be used in a DoS attack. [15] carefully considers the pro's and con's and ends up suggesting a number below 100. Privacy Pass enjoys fast token verification (one PRF evaluation and one hash), double spending protection, and seamless key rotation (server chooses a fresh OPRF key), while the communication and computation of its issuing phase is linear in the number n of requested tokens (n PRF evaluations). Privacy Pass is available through the Chrome and Firefox browsers.

A new approach to building anonymous access tokens. In this work, we build an alternative to Privacy Pass trading faster token issuing on the server-side for slower token verification, which is suitable for scenarios with few issuing servers and many verification servers (cf. 8). In our system, communication as well as server-side computation during the issuing phase is constant and independent of the number of tokens a user gets for each CAPTCHA solution. The constant-size data (called pre-token), received during issuance, enables a user to locally derive (multiple) tokens. Our tokens are further *fine-tunable* in the sense that the server can adaptively decide *how many* tokens any credible user is allowed to derive from the pre-token at any given point in time *after* the CAPTCHA was already solved. For example, a server can decide to allow every credible user to compute another 100 tokens whenever fast content delivery has priority, and it can decide to limit the number of newly created tokens to 1 per credible user whenever there is an increased risk of a bot attack, i.e., security has priority. This redemption policy update does not cause any overhead such as further interactions between issuing server and users. Besides this sort of time-wise fine-tuning, also a content-specific fine-tuning is possible (without having separate instances of our scheme), by allowing a different number of tokens per credible user for different kind of web contents. This could be especially beneficial in avoiding further DoS-Attacks by allowing less tokens for more computation-intensive tasks as API calls. See Section 8 and Appendix D for further details on updatable policies.

As in Privacy Pass, token spendings are anonymous and unlinkable, where we achieve a somewhat weaker form of unlinkability (cf. Section 8), key rotation is seamless, and double spending can be prevented by the server.

1. This is a slight simplification: the Privacy Pass protocol does not send w in the clear, but $H(w, \text{con})$ for some context data con that is known to both the server and the browser. This allows to bind the token spending to a specific context.

A key contribution of our paper is a new cryptographic primitive that we call *group verifiable random function* (GVRF), which is particularly useful for constructing anonymous tokens. Essentially, we can replace the OPRF in Privacy Pass with a GVRF, and directly gain the above-described issuing speedup and fine tuning. In a nutshell, a GVRF allows each user in a group to anonymously evaluate their *own* random function in a verifiable way. More precisely, if Alice and Bob are two group members (each holding a secret key sk_A, sk_B), their evaluations $\text{VRF}_{sk_A}(x)$ and $\text{VRF}_{sk_B}(x)$ of any input x do not reveal anything about sk_A and sk_B . This description is already enough to explain the usefulness of GVRFs in the context of anonymous tokens, and we will give a more comprehensive introduction of GVRFs later. The main idea is to replace the “global” PRF in Privacy Pass with the *user-specific* random functions of a GVRF. This approach makes the following simple but significant difference: we can re-use the user-generated parts t of the tokens among all users. Let us explain this in more detail. Assume tokens are of the form $(t, f(t))$, where $f()$ is a PRF in Privacy Pass, and a GVRF in our work. If $f()$ is a PRF and the server holds the PRF key K as in Privacy Pass, *every user* needs to receive tokens of the form $(t, \text{PRF}_K(t))$ for the same K , as otherwise the server could recognize users by PRF keys. This actually decreases the efficiency of Privacy Pass, where more than half of the user’s computation complexity is for verifying that the server indeed used the “global” PRF key K (cf. Table 1 in [15]), which ensures the user’s anonymity. At the same time, it requires users to choose *random* values t , simply to ensure that nobody else already used t for their token request. On the other hand, our token scheme replaces the PRF by a user-specific VRF as $(t, \text{VRF}_{sk}(t))$, where sk is a user’s VRF evaluation key, which is certified by a group manager playing the role of the issuing server. However, if we would use a standard VRF with user-specific certified public keys for verification, the token system would not be anonymous. *Group* VRFs, as designed in this paper, allow for *anonymous* yet verifiable evaluations within the group of all users who ever got issued such evaluation secret key (i.e., whoever solved a CAPTCHA). Because every user still evaluates their own VRF, we can now re-use t among users! And with that, we can let the server install redemption policies even after a signing key sk was issued. For example, a server can say “I am now accepting tokens of the format $(t = 2024 - 03 - 19_a, \text{VRF}_{sk}(t))$ with $a \in \{1, \dots, 10\}$ ”, thereby limiting the number of tokens that credible users can locally generate for their access requests to 10. On the downside, restricting the first component t of all tokens to come from a small set also causes some leakage: if two tokens with the same t are redeemed, it follows that they must have been generated by different users (cf. Section 8 for further discussions). Besides this new fine-tuning option, token systems from group VRFs achieve constant token issuance complexity on the server-side, because all that needs to be computed is a certified sk .

Our Group VRF Construction. We construct a group VRF by augmenting an asymmetric version of the Dodis-Yampolskiy (DY) VRF [16] with re-randomizable certificates of VRF public keys. Recall that the DY VRF in an

asymmetric bilinear group with a pairing $e : \mathbb{G}_1, \mathbb{G}_2 \rightarrow \mathbb{G}_T$, for a PRF key $sk \in \mathbb{Z}_p$, where $p = |\mathbb{G}_i|$, with corresponding public key $pk := g_1^{sk}$, computes the PRF value of $x \in \mathbb{Z}_p$ as $F_{sk}^{\text{DY}}(x) := e(g_1, g_2)^{1/(x+sk)}$. A proof of correct computation of $F_{sk}^{\text{DY}}(x)$ is given in form of a preimage $\pi := g_2^{1/(x+sk)}$ under the pairing. I.e., the computation was correctly performed with sk if $F_{sk}^{\text{DY}}(x) = e(g_1, \pi)$ and $e(pk \cdot g_1^x, \pi) = e(g_1, g_2)$. We modify the DY PRF in essentially two ways to obtain a group VRF, following the ideas of Ganesh et al. [23] to achieve anonymity through non-uniqueness of public keys.

- 1) Public keys of the form (g_1, g_1^{sk}) are *signed* under a signing key held by the group manager, and such a signature (“certificate”) needs to be attached to each computation and checked upon verification.
- 2) Public keys (g_1, g_1^{sk}) can be randomized to $(g_1^\tau, g_1^{\tau sk})$. In order to preserve correctness, we change the proof to $g_2^{1/(\tau(x+sk))}$, such that the randomization factor τ cancels out during verification.

While this already outlines our construction, we would like to point out two subtleties when adapting the DY VRF to a group VRF. First of all, adapting the standard version of the DY VRF in the symmetric bilinear group setting, i.e., where $F_{sk}^{\text{DY}}(x) = e(g, g)^{1/(x+sk)}$, would lead to a candidate where anonymity could be trivially broken, as public keys $pk = (g, g^{sk})$, $\tilde{pk} = (g^\tau, g^{\tau sk})$ could be linked via checking $e(g, g^{\tau sk}) \stackrel{?}{=} e(g^\tau, g^{sk})$.

Second of all, step (1) requires care. Indeed, we cannot use a standard signature scheme, since a “static” signature on a user public key (g_1, g_1^{sk}) would void anonymity guarantees. Hence, we require a signature scheme where signatures can be randomized and adapted to verify under different randomized messages (i.e., user public keys in our case). Fuchsbauer et al. [20] construct a signature scheme where signatures can be “mauled” to verify under another (random) message in the same equivalence class as the originally signed message. Plugging in their scheme in step (1) immediately yields our group VRF.

Further, it turns out challenging to prove our construction secure. Intuitively, to prove anonymity the reduction has to be able to generate proofs containing $\left(g_1^\tau, g_1^{\tau\alpha}, g_2^{\frac{1}{\tau(x+\alpha)}}\right)$ without knowing whether $\alpha = sk_0$ or $\alpha = sk_1$, which can be viewed as a combination of the decisional Diffie-Hellman (DDH) assumption over $\mathbb{G}_1$ and a bilinear version of the strong decisional Diffie-Hellman inversion assumption (which allows proving pseudorandomness of the DY PRF [16]). However, since the same challenge α is used for both, merely assuming hardness of the two assumptions is not sufficient.

Instead, we prove our construction secure under a new DDH-type assumption. We note that our VRF is pairing-based, has small proofs for correctness of evaluation and does not use “generic” NIZKs (e.g., Groth-Sahai proofs or Fiat-Shamir based Sigma protocols) for those proofs. For such VRFs, Brandt et al. [10] show that building on complex (non-standard) assumptions or relying on idealized models is unavoidable. We prove our new assumption to hold in the generic group model (GGM) [34], [30] instead of the more realistic algebraic group model (AGM) [21], [32], since the latter seem to only allow a reduction to a *non-standard* DL-type problem

in our case. In fact, as a by-product we prove hardness in the GGM for a very broad class of decisional problems in bilinear settings, encompassing Uber-problems [9] as special instance, which might be of independent interest.

Naturally, the existence of a group manager comes with the question of which additional power the group manager has, compared to group members. For building anonymous token system, we require that the group manager cannot break any of the pseudorandomness or anonymity properties that group members enjoy. In particular, we require that group managers, just as group members, cannot lift the anonymity of honest members, link their evaluations, or falsely accuse them of having computed an evaluation. Moreover, our group VRF needs to be *dynamic* in the sense that it does not require to update the group public key whenever members join the group, and it guarantees security in the presence of maliciously generated public keys. We give formal definitions for (dynamic) group VRFs and all its desirable properties, and demonstrate that our group VRF satisfies them.

Instantiating our GVRF with the structure-preserving signature scheme of [20] results in the following efficiency: A user public key including its certificate consists of 4 $\mathbb{G}_1$ and 1 $\mathbb{G}_2$ elements. Generating this certificate requires 3 exponentiations in $\mathbb{G}_1$ and 1 in $\mathbb{G}_2$. Verifying it takes 5 pairing evaluations. Computing a GVRF evaluation costs one $\mathbb{G}_T$ exponentiation, computing the VRF proof requires 4 $\mathbb{G}_1$ and 2 $\mathbb{G}_2$ exponentiations. The resulting proof consists of 4 $\mathbb{G}_1$ and 2 $\mathbb{G}_2$ elements. Verifying a GVRF evaluation takes 7 pairing evaluations and 1 $\mathbb{G}_1$ exponentiation.

Related Work. Anonymous VRFs [23] produce verifiable but anonymous pseudorandomness by allowing to transform the verification key in such a way that it still allows for verification, but cannot be linked to other public keys. Evaluation keys can however be generated by anybody, and hence anonymous VRFs do not directly lend themselves to building anonymous token schemes. Our construction of group VRFs follows the idea of [23] with public key transformation, but additionally introduces a group manager that allows to control the overall set of users that can evaluate the VRF – without breaking the anonymity guarantees. Another related concept are *unique group signatures* [19], which mandate a group manager to control the group, but which do not preserve the anonymity of group members in front of this manager. We provide a more detailed comparison of GVRFs, anonymous VRFs and unique blind signatures in Appendix A.

Building on Privacy Pass [15], several recent works design *anonymous token* systems. Kreuter et al. [29] built anonymous tokens with so-called private metadata bit. These tokens embed a single private bit that is accessible only to the verifying authority. Their construction is DDH-based and uses non-interactive zero-knowledge proofs, which can be dispensed with at the cost of a weak correctness notion. Their construction can be viewed as modifying the oblivious PRF underlying Privacy Pass, the so-called 2Hash Diffie Hellman protocol, to achieve the desirable properties.

Chase et al. [13] revisits the definitions of hidden metadata bits of Kreuter et al. [29] and provides stronger versions, from algebraic MACs instead of PRFs.

Benhamouda et al. [5] build similar token system as Kreuter et al. but with *public* verification, essentially combining [29] with blind Schnorr signatures. Similarly, Silde and Strand [35] build anonymous token systems with *public* metadata that allows to add, e.g., an expiration date to tokens, and that are publicly verifiable. Such tokens can find adoption in contact tracing. Their work also suggests batch revocation methods for Privacy Pass and the token scheme of Kreuter et al., which can be used instead of key rotation at the token verifier (which would be arguably problematic in the case of public verifiability). The anonymous token schemes in our work can have both private and public verifiability.

Chu et al. [14] formalise *rate-limited* Privacy Pass, as currently being standardised in IRTF [26], with two new entities called *mediator* and *issuer* to enforce rate-limiting. The anonymity trust assumption is that both parties do not collude. Instead of working with a vOPRF, a blinded signature scheme is used to sanitise tokens for anonymity. While such rate limiting is not the focus of our work, our scheme could be extended with an authentication scheme during token issuance to enforce rate limiting while still keeping verification anonymous without additional communicational overhead. The disadvantage here is that rate limiting always applies to a set of users.

Potentially closest to our work is the token scheme by Benhamouda et al. [6], which builds so-called *counting* tokens. These tokens allow the verifier to count how many different users obtained a token for a particular context. Formally, this is achieved by (1) different users producing different tokens (as opposed to the one PRF in PrivacyPass), and (2) adding a rate limit that disallows the same user to successfully redeem two tokens on the same message. In our work, we follow a similar strategy and hence our token are also counting. However, while Benhamouda et al. still use an OPRF (the Boneh-Boyen PRF) and require communication per issued token, we replace the OPRF by a group VRF which naturally yields (1) and (2). On top, our GVRF-based token scheme allows to adaptively decide on messages, while their construction requires token contents to be decided prior to online issuance.

Finally, our approach of making token generation local by having a PRF key signed instead of the tokens itself bears resemblance with the construction of compact e-cash [12].

Outline of this paper. Section 3 introduces the concept of GVRF. GVRF-based anonymous tokens are explained in Section 4. Required building blocks and assumptions for GVRFs are described in Section 5. Section 6 presents our GVRF construction. Finally, Section 7 presents our benchmarks and Section 8 deployment considerations.

2. Notation

We will use the following notation. By $\lambda \in \mathbb{N}$ we denote the security parameter. By $x \xleftarrow{\$} S$ we denote the process of sampling an element x from set S uniformly at random. By $y \leftarrow x$ we denote the process of assigning y the value of x . We say a function is negligible in λ , if

its inverse vanishes asymptotically faster than any polynomial in λ . We say that an algorithm A is probabilistic polynomial time (PPT), if A is a probabilistic algorithm with running time polynomial in its input length. We use $y \leftarrow A(x)$ to denote that y is assigned the output of A running on input x .

We consider the adversary in security experiments to be stateful, e.g., keeping all state from prior runs. When writing $\mathcal{A}: \perp$ in a non-interactive method, we denote an adversary arbitrarily deviating from the protocol description.

3. Group-Verifiable Random Functions

Our definition of group VRFs is inspired by Groth’s dynamic group signatures [24]. We first present the plain definition, which only requires correctness, pseudorandomness and unique provability. Subsequently, we define additional notions such as (weak) unlinkability and unique opening.

In our definition of group verifiable random function, we will assume a group manager (also sometimes referred to as issuer), who can decide who can join the group.

Definition 1 (Group Verifiable Random Function). A *group verifiable random function* (GVRF) is a tuple of PPT algorithms defined as follows.

- $pp \leftarrow \text{Setup}(1^\lambda)$ is an algorithm to set up the public parameters pp including the inputs space $\mathcal{X}$, the output space $\mathcal{Y}$. In the following we assume all algorithms to have access to pp without stating it explicitly.
- $(pk_G, sk_G) \leftarrow \text{GroupKG}(pp)$ is a probabilistic algorithm (run by the issuer) that outputs the group public key pk_G and the group secret key sk_G , where the latter is kept private by the issuer.
- $b \leftarrow \text{VerGroup}(pk_G)$ is an algorithm that takes as input a group public key pk_G and outputs a bit $b \in \{0, 1\}$.
- $(pk, sk) \leftarrow \text{KG}(pk_G)$ is a probabilistic algorithm run by a user that on input of the group public key pk_G , outputs a key pair (pk, sk) .
- $\text{crt} \leftarrow \text{Join}(sk_G, pk)$ is a PPT algorithm executed by the issuer on input sk_G and user public key pk . The algorithm outputs a certificate crt .
- $b \leftarrow \text{VerCert}(pk_G, pk, \text{crt})$ is a deterministic algorithm that on input of the group public key pk_G , the user public key pk and the certificate crt , outputs a bit $b \in \{0, 1\}$.
- $(y, \pi, \tau) \leftarrow \text{Eval}(pk_G, pk, sk, \text{crt}, x)$ is an algorithm that on input of the group public key pk_G , the user public key pk and secret key sk , the user value $x \in \mathcal{X}$, and the certificate crt , outputs a value $y \in \mathcal{Y}$, a proof π , and opening information τ .
- $b \leftarrow \text{Ver}(pk_G, x, y, \pi)$ is an algorithm that on input of the group public key pk_G , an input-value pair $x \in \mathcal{X}$, $y \in \mathcal{Y}$ and a proof π and outputs a bit b .
- $b \leftarrow \text{Judge}(pk_G, pk, x, y, \pi, \tau)$ is an algorithm that takes as input the group public key pk_G , a user public key pk , an input-value pair $x \in \mathcal{X}$, $y \in \mathcal{Y}$, a proof π , an opening information τ , and outputs a bit b .

A GVRF must satisfy correctness, pseudorandomness and unique provability defined below.

We discuss several aspects of the definition, in relation to the definitions of group signatures. As in [24], our GVRF is “dynamic” in the sense that users can join the group at any time, without the need to update the group public key pk_G . Opposed to signature verification, algorithm Ver cannot take a user public key as input, since this would violate the desirable anonymity within the group. Hence, Ver needs to verify from only the group public key. Finally, users can decide to lift their anonymity using the Judge algorithm. In this work we focus on this user-centric flavor of deanonymization, but note that other approaches are possible (e.g., where the group manager is given the capability to trace users [24]).

3.1. Basic requirements on GVRFs

In the following we give basic requirements on our GVRF. Note that these correspond essentially to the standard VRF properties lifted to the group setting. In particular, note that our definition of unique verifiability essentially allows to recover the standard notion of unique verifiability by using Judge as verification algorithm. Note that in the following we give the definition relative to a single public key/ secret key pair (pk, sk) , which implies that correctness holds for every honestly generated key pair in a group.

Definition 2 (Correctness of a GVRF). A GVRF $(\text{Setup}, \text{GroupKG}, \text{VerGroup}, \text{KG}, \text{Join}, \text{VerCert}, \text{Eval}, \text{Ver}, \text{Judge})$ is *correct* if it satisfies the following conditions for all security parameters $\lambda \in \mathbb{N}$, for all $\{\mathcal{X}, \mathcal{Y}\} \subseteq pp$ in the image of $\text{Setup}(1^\lambda)$, for all (pk_G, sk_G) in the image of $\text{GroupKG}(pp)$ for all (pk, sk) in the image of $\text{KG}(pp, 1^\lambda)$ and $\text{crt} \leftarrow \text{Join}(pp, sk_G, pk)$:

- *Correctness of certificates:* It holds $\text{VerGroup}(pk_G) = 1$ and $\text{VerCert}(pk_G, pk, \text{crt}) = 1$.
- *VRF correctness:* For all $x \in \mathcal{X}$, for all (y, π, τ) in the image of $\text{Eval}(pk_G, pk, sk, \text{crt}, x)$, it holds $\text{Ver}(pk_G, x, y, \pi) = 1$.
- *Opening correctness:* For all $x \in \mathcal{X}$, for all (y, π, τ) in the image of $\text{Eval}(pk_G, pk, sk, \text{crt}, x)$, it holds $\text{Judge}(pk_G, x, y, \tau, \pi) = 1$.

We next formalize pseudorandomness of GVRF. The standard notion of pseudorandomness for VRFs is extended to the group setting in the following way: outputs of the GVRF need to appear pseudorandom to every member of the group, and to the group manager holding sk_G .

Definition 3 (Pseudorandomness). A GVRF $(\text{Setup}, \text{GroupKG}, \text{VerGroup}, \text{KG}, \text{Join}, \text{VerCert}, \text{Eval}, \text{Ver}, \text{Judge})$ satisfies *pseudorandomness*, if for all admissible PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}: \mathbb{N} \rightarrow \mathbb{R}_{>0}$, such that for all $\lambda \in \mathbb{N}$ it holds

$$\Pr[\text{Exp}_{\text{GVRF}, \mathcal{A}}^{\text{pseudorandom}}(1^\lambda) = 1] - \frac{1}{2} \leq \text{negl}(\lambda),$$

where $\text{Exp}_{\text{GVRF}, \mathcal{A}}^{\text{pseudorandom}}(1^\lambda)$ is as defined in Figure 1, where we say an adversary is admissible if it

$\text{Exp}_{\text{GVRF}, \mathcal{A}}^{\text{pseudorandom}}(1^\lambda):$ $\{\mathcal{X}, \mathcal{Y}\} \subseteq pp \leftarrow \text{Setup}(1^\lambda)$ $pk_G \leftarrow \mathcal{A}(pp)$ $Q \leftarrow \emptyset, (sk, pk) \leftarrow \text{KG}(pk_G),$ $\text{crt} \leftarrow \mathcal{A}(pk)$ $x^* \leftarrow \mathcal{A}^{\text{Eval}(\cdot, pk, sk, \text{crt}, \cdot)}(1^\lambda)$ $(y, \pi, \tau) \leftarrow \text{Eval}(pk_G, pk, sk, \text{crt}, x^*)$ $y_0 \leftarrow y, y_1 \xleftarrow{\$} \mathcal{Y}, b \xleftarrow{\$} \{0, 1\}$ $b^* \leftarrow \mathcal{A}^{\text{Eval}(\cdot, y_0, sk, \text{crt}, \cdot)}(y_0)$ return $b = b^* \wedge x^* \notin Q$	$\mathcal{O}_{\text{Eval}}(x, \text{crt}):$ $Q = Q \cup \{x\}$ $(y, \pi, \tau) \leftarrow \text{Eval}(pk_G, pk, sk, \text{crt}, x)$ return (y, π)
--	---

Figure 1: Pseudorandomness experiment for GVRF. The adversary wins if he can distinguish an evaluation of self-chosen x^* done with respect to some pk from a randomly chosen element from the image space, where only adversaries which provide a valid public key pk_G and valid certificates crt relative to pk are considered.

provides pk_G and crt such that $\text{VerGroup}(pk_G) = \text{VerCert}(pk_G, pk, \text{crt}) = 1$, and where the randomness is taken over the random coins of Setup, Join and $\mathcal{A}$, as well as the random choices of the bit b and image y_1 .

Next we define unique provability. Recall that unique provability is a property demanded from VRFs. It demands that it is hard to produce two distinct VRF values of the same input both with verifying proofs *relative to the same public key*. Note that we cannot require unique provability relative to the verification Ver, since Ver is defined relative to a group manager, and thus a potential group of public keys.² Once keys are opened via Judge to a unique public key though, we can require the usual unique provability. Namely:

Definition 4 (Unique Provability). We say a GVRF $(\text{Setup}, \text{GroupKG}, \text{VerGroup}, \text{KG}, \text{Join}, \text{VerCert}, \text{Eval}, \text{Ver}, \text{Judge})$ satisfies *unique provability*, if for all $\lambda \in \mathbb{N}$, all public parameters pp in the image of $\text{Setup}(1^\lambda)$, for all (pk_G, sk_G) in the image of $\text{GroupKG}(pp)$, for all possible public keys pk (i.e., potentially maliciously generated), for all possible certificates crt, for all possible input values x , for all possible function values y_0, y_1 for all possible proofs π_0, π_1 and for all possible opening values τ_0, τ_1 it holds the following: if $\text{Judge}(pk_G, pk, x, y_0, \pi_0, \tau_0) = \text{Judge}(pk_G, pk, x, y_1, \pi_1, \tau_1) = 1$, then $y_0 = y_1$. (In other words, for each $x \in \mathcal{X}$ there exists at most one possible function value y to which x can be opened to under pk . By correctness it is exactly on y , whenever pk is generated honestly.)

3.2. Group-bounded provability

Recall that in the group setting an adversary could have potentially corrupted members of the group, and hence is in possession of an arbitrarily large fraction of the secret keys. For an adversary holding n secret keys (relative to some group defined by pk_G), it is thus easy to produce n images with verifying proofs for the same input value $x \in \mathcal{X}$. We capture the intuition that an adversary should not be able to produce *more* valid images of x (relative to pk_G) in the notion of group-bounded provability.

2. To make the verification algorithm Ver meaningful for applications, we introduce the notion of “group-bounded provability” below.

$\text{Exp}_{\text{GVRF}, \mathcal{A}}^{\text{gb-prov}}(1^\lambda):$ $pp \leftarrow \text{Setup}(1^\lambda), \text{ctr} := 0$ $(pk_G, sk_G) \leftarrow \text{GroupKG}(pp)$ $(x^*, y_1, \dots, y_n, \pi_1, \dots, \pi_n) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Join}}(\cdot)}(pp, pk_G)$ if $[\forall i \neq j: y_i \neq y_j] \wedge [\forall i: \text{Ver}(pk_G, x^*, y_i, \pi_i) = 1]$ $\wedge \text{ctr} < n$ return 1 else return 0	$\mathcal{O}_{\text{Join}}(pk):$ $\text{crt} \leftarrow \text{Join}(sk_G, pk)$ $\text{ctr} \leftarrow \text{ctr} + 1$ return crt
---	--

Figure 2: Group-bounded provability experiment for GVRF. The adversary wins if it can provide at least one more evaluation of self-chosen x^* than it possesses secret keys within the group.

$\text{Exp}_{\text{GVRF}, \mathcal{A}}^{(\text{weak-})\text{unlink}}(1^\lambda):$ $\{\mathcal{X}, \mathcal{Y}\} \subseteq pp \leftarrow \text{Setup}(1^\lambda)$ $pk_G \leftarrow \mathcal{A}(pp)$ $\mathcal{Q} := \emptyset, \mathcal{Q}_{\text{open}} := \emptyset$ $(pk_0, sk_0) \leftarrow \text{KG}(pk_G)$ $(pk_1, sk_1) \leftarrow \text{KG}(pk_G)$ $(\text{crt}_0, \text{crt}_1) \leftarrow \mathcal{A}(pk_0, pk_1)$ $b \xleftarrow{\$} \{0, 1\}$ $b^* \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Eval}}^0(\cdot), \mathcal{O}_{\text{Eval}}^1(\cdot), \mathcal{O}_{\text{Eval}}^0(\cdot)}(1^\lambda)$ return $(b == b^*) \wedge (\mathcal{Q} \cap \mathcal{Q}_{\text{open}} == \emptyset)$	$\mathcal{O}_{\text{Eval}}^b(x):$ $\mathcal{Q} := \mathcal{Q} \cup \{x\}$ $(y, \pi, \tau) \leftarrow \text{Eval}(pk_G, pk_b, sk_b, \text{crt}_b, x)$ return (y, π) $\mathcal{O}_{\text{Eval}}^0(x):$ $\mathcal{Q}_{\text{open}} := \mathcal{Q}_{\text{open}} \cup \{x\}$ $(y, \pi, \tau) \leftarrow \text{Eval}(pk_G, pk_0, sk_0, \text{crt}_0, x)$ return (y, π) $\mathcal{O}_{\text{Eval}}^1(x):$ $\mathcal{Q}_{\text{open}} := \mathcal{Q}_{\text{open}} \cup \{x\}$ $(y, \pi, \tau) \leftarrow \text{Eval}(pk_G, pk_1, sk_1, \text{crt}_1, x)$ return (y, π)
--	--

Figure 3: (Weak-)Unlinkability experiment for GVRF. The adversary needs to recognize which of two public keys is used for a series of evaluations of self-chosen inputs. In the full unlinkability experiment, the adversary additionally has access to the oracles $\mathcal{O}_{\text{Eval}}^0, \mathcal{O}_{\text{Eval}}^1$ highlighted in red. Here, we only consider adversaries which provide a valid public key pk_G and valid certificates crt_β relative to pk_β for $\beta \in \{0, 1\}$.

Definition 5 (Group-bounded provability). We say a GVRF $(\text{Setup}, \text{GroupKG}, \text{VerGroup}, \text{KG}, \text{Join}, \text{VerCert}, \text{Eval}, \text{Ver}, \text{Judge})$ has *group-bounded provability* if for all PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}: \mathbb{N} \rightarrow \mathbb{R}_{>0}$, such that for all $\lambda \in \mathbb{N}$ it holds

$$\Pr[\text{Exp}_{\text{GVRF}, \mathcal{A}}^{\text{gb-prov}}(1^\lambda) = 1] \leq \text{negl}(\lambda),$$

where $\text{Exp}_{\text{GVRF}, \mathcal{A}}^{\text{gb-prov}}(1^\lambda)$ is as defined in Figure 2 and the randomness is taken over the random coins of Setup, Join, and $\mathcal{A}$.

3.3. (Weak) Unlinkability

Next, we define two flavors of unlinkability. Weak unlinkability essentially demands that evaluations of different preimages should not reveal by which group member they were computed. If this holds even if the adversary has already seen an arbitrary set of disjoint evaluations of the group members in question, we refer to it as (full) unlinkability.

Definition 6 ((Weak) Unlinkability). A GVRF $(\text{Setup}, \text{GroupKG}, \text{VerGroup}, \text{KG}, \text{Join}, \text{VerCert}, \text{Eval}, \text{Ver}, \text{Judge})$ satisfies *(weak) unlinkability*, if for all admissible PPT adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}: \mathbb{N} \rightarrow \mathbb{R}_{>0}$, such that for all $\lambda \in \mathbb{N}$ it holds

$$\Pr[\text{Exp}_{\text{GVRF}, \mathcal{A}}^{(\text{weak-})\text{unlink}}(1^\lambda) = 1] - \frac{1}{2} \leq \text{negl}(\lambda),$$

where $\text{Exp}_{\text{GVRF}, \mathcal{A}}^{(\text{weak-})\text{unlink}}(1^\lambda)$ is as defined in Figure 3, where we say an adversary is admissible if it provides pk_G and $\text{crt}_0, \text{crt}_1$ such that $\text{VerGroup}(pk_G) = 1$ and $\text{VerCert}(pk_G, pk_0, \text{crt}_0) = \text{VerCert}(pk_G, pk_1, \text{crt}_1) = 1$, and the randomness is taken over the random coins of Setup and $\mathcal{A}$, as well as the random choices of the bit b and image y_1 .

3.4. Unique opening

Next we define unique opening, which prevents malicious users from claiming ownership of an input/output pair x, y which was evaluated by an honest user under pk relative to a maliciously generated public key $pk^* \neq pk$.

Definition 7 (Unique Opening). We say a group VRF $\text{GVRF} = (\text{Setup}, \text{GroupKG}, \text{VerGroup}, \text{KG}, \text{Join}, \text{VerCert}, \text{Eval}, \text{Ver}, \text{Judge})$ satisfies *unique opening*, if for all $\lambda \in \mathbb{N}$, all public parameters pp in the image of $\text{Setup}(1^\lambda)$, for all (pk_G, sk_G) in the image of $\text{GroupKG}(pp)$, for all possible public keys pk_0, pk_1 , for all possible certificates $\text{crt}_0, \text{crt}_1$, for all possible input values x , for all possible function values y for all possible proofs π_0, π_1 and for all possible opening values τ_0, τ_1 it holds that: if $\text{Judge}(pk_G, pk_0, x, y, \pi_0, \tau_0) = \text{Judge}(pk_G, pk_1, x, y, \pi_1, \tau_1) = 1$, then $pk_0 = pk_1$. (In other words, for each input/output pair $(x, y) \in \mathcal{X} \times \mathcal{Y}$ there exists at most one possible public key pk such that (x, y) can be opened under pk .)

Note that for the use in anonymous token schemes the ability to de-anonymize oneself is not required, one could thus also define GVRFs without opening information τ , without a Judge algorithm, and without requiring *unique opening*. Since we believe that the possibility to “claim” an evaluation is useful in contexts like lottery schemes, where one might want to claim a lottery win, and since our instantiation naturally satisfies this additional requirement, we opted for the more general definition.

3.5. Power of the group manager

We want to stress that the only power our group manager has is to decide who can join the group by generating a certificate. He cannot, however, evaluate the VRF relative to a group member (pseudorandomness), link a VRF evaluation to a group member (unlinkability), or frame a group member for a (malicious) evaluation (pseudorandomness & unique provability).

4. Anonymous Tokens from Group VRFs

GVRFs are useful tools for building anonymous access token systems. We now give an algorithmic description of a *policy-based anonymous token scheme*. We aim at a general definition that can capture previous constructions such as Privacy Pass, but that is also suitable to demonstrate particular aspects of GVRF-based access tokens.

- $\text{AT.Setup}(1^\lambda)$ is a global and trusted setup that is run once to generate the (re-usable) public parameters pp .

All parties and algorithms are assumed to have access to pp .

- $\text{AT.ServerSetup}(S: pp)$ is run by S once and outputs a secret key sk_S to the server, and the public key pk_S to all parties.
- $\text{AT.VerSSetup}(U: pk_S)$ is run by a user (or another entity advocating privacy) to verify the server setup. It outputs a bit $b \in \{0, 1\}$.
- $\text{AT.UpdatePolicy}(S: \{p_1, \dots, p_n\})$ for any $n \in \mathbb{N}$ is run by S and sets $\mathcal{P} \leftarrow \{p_1, \dots, p_n\}$ or outputs $\perp$.
- $\text{AT.Generate}(U: pk_S; S: sk_S)$ is run between a user U and S . It outputs a pre-token pre or $\perp$ to user U .
- $\text{AT.Expand}(U: p, \text{aux}, \text{pre})$ is an algorithm run by the user U that, on input a pre-token pre , auxiliary information aux and an element p from the policy $\mathcal{P}$ outputs a token t and a proof π .
- $\text{AT.Verify}(V: t, \text{aux}, \pi, \mathcal{P}, sk_S | pk_S)$ is a (stateful) algorithm run by V that outputs a bit b . If it is sufficient that the verifier inputs pk_S instead of sk_S we call the scheme *publicly verifiable*.

Policy-based anonymous token scheme. A policy-based anonymous token scheme (pbATS) is run with arbitrarily many users, one issuing server (S in the below) that grants access based on the policy and generated (pre-)tokens, and a verifier V . A pbATS adds a global policy to standard anonymous token schemes that allows the verifier to adaptively control the number of tokens that can be successfully redeemed by a credible user. To model the adaptivity, users first obtain a so-called *pre-token* pre from the issuing server (AT.Generate). According to the current global policy, they can then locally expand that pre-token in several actual tokens (AT.Expand), optionally including auxiliary information aux in the token, e.g., to bind it to a particular context. A change in policy can then allow users to compute more tokens even without interacting with the issuer again. The policy hence allows the verifier to set the security level: a strict policy $\mathcal{P} = \{p_1\}$ allows each user to extract exactly one token from each pre-token, requiring to again contact the issuing server if they want more tokens. A relaxed policy $\mathcal{P} = \{p_1, \dots, p_n\}$ allows to expand each pre-token into n access tokens, taking off the load from the issuing server and hence enabling faster access to resources.

Intuitively, we want $\text{AT.Verify}(V: t, \text{aux}, \pi, \mathcal{P}, sk_S | pk_S)$ to output 1 if and only if there exists a policy $p \in \mathcal{P}$ and a pre-token pre generated by $\text{AT.Generate}(U: pk_S, S: sk_S)$ such that (t, π) was an output of $\text{AT.Expand}(U: p, \text{aux}, \text{pre})$ and t has not been used previously. In particular, we want *correctness* (i.e., a honestly generated token will be accepted by the verifier), *unforgeability* (i.e., a user holding k pre-tokens cannot generate more than $k \cdot |\mathcal{P}|$ accepting tokens relative to $\mathcal{P}$) and *unlinkability* (i.e., a server cannot link a pre-token pre with a token t).

Before formally defining these properties, we will explain how Privacy Pass can be viewed as a pbATS. Recall that Privacy Pass is based on a verifiable oblivious pseudorandom function (vOPRF). A vOPRF is a two-party primitive, where a server holds a secret key k , such that the client (user) can evaluate the pseudorandom function $f_k(\cdot)$ on inputs x of her choice, such that the server does not learn anything about the choice x , and the client does not learn anything about the secret key k (except the output

$f_k(x)$). Verifiability further guarantees that the client can verify that $f_k(x)$ was correctly evaluated. Now, to obtain a token in Privacy Pass, the user receives $y = f_k(x)$ for an x of her choice. The user then computes $\pi := \text{MAC}_{k'}(\text{aux})$ for $k' := \text{KDF}(y)$, where KDF is some key derivation function that allows to derive a key k' for a message authentication code MAC.³ To redeem, the user can send (x, π) to the server, which accepts if and only if x is fresh (i.e., has not been queried before) and it can recompute π from x, k and aux . Intuitively, we have unforgeability because of the pseudorandomness of $f_k(\cdot)$ (i.e., generating an accepting token would correspond to guessing the output value on a fresh x') and unlinkability because the server did not learn anything about x at the time of the token generation. With this, it is straightforward to obtain a pbATS. as follows.

- $\text{PPass.Setup}(1^\lambda)$ generates public parameters for the vOPRF used by Privacy Pass.
- $\text{PPass.ServerSetup}(S: pp)$ runs the vOPRF setup and returns an vOPRF key $sk_S = k$ to the server and a public commitment pk_S to the key k to all parties.
- $\text{PPass.VerSSetup}(U: pk_S)$ is not explicitly supported by Privacy Pass, instead a correct server setup (knowledge of k) is implicitly checked as part of the proof for a valid pre-token.
- Privacy Pass does not support $\text{AT.UpdatePolicy}(S: \mathcal{P})$.
- $\text{PPass.Generate}(U: pk_S; S: sk_S)$ invokes a vOPRF evaluation between the user U and the server S , where the user inputs a randomly chosen message x and the server inputs the vOPRF key $sk_S = k$, and the user obtains the pre-token $(x, f_k(x))$. If the output does not verify, the user outputs $\perp$. Proofs of multiple vOPRF evaluations are batchable.
- $\text{PPass.Expand}(U: \perp, \text{aux}, \text{pre})$ parses $\text{pre} = (x, f_k(x))$, returns the token $t = x$ and proof $\pi = \text{MAC}_{\text{KDF}(y)}(\text{aux})$, i.e., each pre-token gives exactly one token.⁴
- $\text{PPass.Verify}(V: t, \text{aux}, \pi, \mathcal{P}, sk_S)$ is run by S , who checks if $\pi = \text{MAC}_{\text{KDF}(f_k(t))}(\text{aux})$ for $k = sk_S$ and t has not been used (note that the policy $\mathcal{P}$ is ignored).

Because each verifying token t requires one interaction with the server, Privacy Pass does not allow the verifier to install policies that would allow to locally generate more tokens through Expand. Furthermore, since access to $sk_S = k$ is required in AT.Verify , Privacy Pass requires $S = V$ to be set, i.e. it is not publicly verifiable. There is an extension that uses a blinded signature scheme instead of a vOPRF to make Privacy Pass publicly verifiable [36]. We note that batching techniques can be applied to render the issuance of n tokens to significantly be more bandwidth-efficient and verifier-efficient than n times the cost of Generate, particularly through batching the proof of correct PRF evaluation in the vOPRF. We will later

3. Note that for the purpose of obtaining a pbATS, one could also consider a simplified version of Privacy Pass without auxiliary input aux , where we set $\pi = f_k(x)$.

4. Note that one can derive many distinct tuples (x, π_i) from a pre-token pre by using different auxiliary inputs aux_i , but since they all share the same x these would be linkable by the server. To avoid double-spending the server checks freshness of $t = x$.

take these techniques into account when comparing our *pbATS* to Privacy Pass.

We now explain how a *pbATS* AT_{GVRF} with a flexible policy can be obtained based on group VRFs. The construction is pretty straightforward: pre-tokens are GVRF user key pairs certified by S who takes the role of the GVRF group manager. Tokens are GVRF evaluations of elements x from the policy under the user secret key. This allows users to locally generate as many tokens as the policy has elements. Token verification equals the verification of a GVRF evaluation, and expiration of all issued pre-tokens and tokens works by generating a fresh GVRF group key pair. More formally, let GVRF denote a group VRF. Then we define a *pbATS* as follows.

- $\text{AT}_{\text{GVRF}}.\text{Setup}(1^\lambda)$ outputs $pp \leftarrow \text{GVRF}.\text{Setup}(1^\lambda)$, where pp includes a description of the input set $\mathcal{X}$ of the GVRF.
- $\text{AT}_{\text{GVRF}}.\text{ServerSetup}(S: pp)$ generates a key pair $(pk_S, sk_S) \leftarrow \text{GVRF}.\text{GroupKG}(pp)$, and outputs sk_S to S and pk_S to all users.
- $\text{AT}_{\text{GVRF}}.\text{VerSSetup}(U: pk_S)$ returns the output of $\text{GVRF}.\text{VerGroup}(pk_S)$.
- $\text{AT}_{\text{GVRF}}.\text{UpdatePolicy}(S: X)$ for any $X \subset \mathcal{X}$ sets $\mathcal{P} \leftarrow X$.
- To run $\text{AT}_{\text{GVRF}}.\text{Generate}(U: pk_S, S: sk_S)$, the user generates a key pair $(pk, sk) \leftarrow \text{GVRF}.\text{KG}(pk_S)$ and sends pk to the server. The server generates $\text{crt} \leftarrow \text{Join}(sk_S, pk)$ and sends crt to the user. The user checks if the obtained certificate verifies via $\text{VerCert}(pk_S, pk, \text{crt}) = 1$, if yes it outputs $\text{pre} = (pk_S, sk, \text{crt})$, otherwise it outputs $\perp$.
- To evaluate $\text{AT}_{\text{GVRF}}.\text{Expand}(U: x, \perp, \text{pre})$, the user parses the pre-token $\text{pre} =: (pk_S, sk, \text{crt})$, computes the GVRF evaluation $(y, \pi, \tau) \leftarrow \text{Eval}(pk_S, pk, sk, \text{crt}, x)$ and outputs token $t := (x, y)$ and proof π .
- $\text{AT}_{\text{GVRF}}.\text{Verify}(V: t, \perp, \pi, \mathcal{P}, pk_S)$ parses $t =: (x, y)$ and outputs 1 if $x \in \mathcal{P}$ and $\text{Ver}(pk_S, x, y, \pi) = 1$ and t has not been used, and 0 otherwise.

Before proving that this indeed yields a *pbATS* with flexible policy, we formally define desirable properties of a *pbATS*.

Correctness. For correctness, we require that *Expand* always produces verifying tokens when run with a fresh element from the policy, a pre-token generated by *Generate*, an arbitrary auxiliary information. More formally:

Definition 8 (Correctness and freshness of *pbATS*).

We say that a *pbATS* $\text{AT} = (\text{Setup}, \text{ServerSetup}, \text{VerSSetup}, \text{UpdatePolicy}, \text{Generate}, \text{Expand}, \text{Verify})$ is *correct*, if for all $\lambda \in \mathbb{N}$, $pp \leftarrow \text{Setup}(1^\lambda)$, for all $(pk_S, sk_S) \leftarrow \text{ServerSetup}(S: pp)$, we have that $\text{VerSSetup}(U: pk_S) = 1$, and for all possible policies $\mathcal{P}$ in the image of *UpdatePolicy*, for all policy elements $p \in \mathcal{P}$, for all pre-tokens $\text{pre} \leftarrow \text{AT}.\text{Generate}(U: pk_S, S: sk_S)$, we have that each (t, π) in the image of $\text{AT}.\text{Expand}(U: p, \text{pre})$ will pass the verification, i.e., it holds that $\text{AT}.\text{Verify}(V: t, \pi, \mathcal{P}, pk_S) = 1$.

We further require *freshness* of the generated token: If $(t, \pi) \leftarrow \text{AT}.\text{Expand}(U: p, \text{pre})$ and $(t', \pi') \leftarrow \text{AT}.\text{Expand}(U: p', \text{pre}')$ with $p \neq p'$ or $\text{pre} \neq \text{pre}'$

$\text{Exp}_{\text{AT}, \mathcal{A}}^{\text{unforgeable}}(1^\lambda):$ $\text{ctr} := 0, pp \leftarrow \text{Setup}(1^\lambda)$ $(pk_S, sk_S) \leftarrow \text{ServerSetup}(S: pp)$ $(\mathcal{P}, (t_i, \pi_i, \text{aux}_i)_{i \in [q]}) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Gen}}}(\text{pp}, pk_S)$ if $[\forall i \neq j: t_i \neq t_j] \wedge \text{ctr} \cdot \mathcal{P} < \ell \wedge [\forall i: \text{Ver}(V: t_i, \text{aux}_i, \pi_i, \mathcal{P}, sk_S) = 1]$ return 1 else return 0	$\mathcal{O}_{\text{Gen}}():$ $\text{pre} \leftarrow \text{AT}.\text{Generate}(\mathcal{A}: \perp, S: sk_S)$ $\text{ctr} \leftarrow \text{ctr} + 1$ return pre
--	---

Figure 4: Unforgeability experiment for anonymous token schemes.

(for $p, p' \in \mathcal{P}$ and honestly generated pre, pre'), we require $t \neq t'$, except with negligible probability.

Correctness of our GVRF-based token scheme AT_{GVRF} immediately follows from the correctness of the GVRF. If the output space $\mathcal{Y}$ of GVRF is sufficiently large (i.e., of superpolynomial size), then freshness follows from the pseudorandomness of the GVRF.

A note on freshness. Note that when redeeming a token the server only checks if a token has already been spent (which by freshness does not happen for a newly generated token, except with negligible probability). If a user's token has not been spent yet, an adversary getting hold of it may spend it. This is a trade-off for more efficiency, since proving freshness would require an interactive protocol (involving a server challenge). Instead, our tokens are static objects (as in Privacy Pass) and enable faster redemption.

Unforgeability. We require that it is hard to produce a verifying token beyond what the policy allows to expand from pre-tokens obtained from the issuer. More formally:

Definition 9 (Unforgeability *pbATS*). We say a *pbATS* $\text{AT} = (\text{Setup}, \text{ServerSetup}, \text{VerSSetup}, \text{UpdatePolicy}, \text{Generate}, \text{Expand}, \text{Verify})$ is *unforgeable*, if for all PPT adversaries $\mathcal{A}$ there exists a negligible function $\text{negl}: \mathbb{N} \rightarrow \mathbb{R}_{>0}$, such that for all $\lambda \in \mathbb{N}$ it holds

$$\Pr[\text{Exp}_{\text{AT}, \mathcal{A}}^{\text{unforgeable}}(1^\lambda) = 1] \leq \text{negl}(\lambda),$$

where $\text{Exp}_{\text{AT}, \mathcal{A}}^{\text{unforgeable}}(1^\lambda)$ is as defined in Figure 4.

Unforgeability ensures that cooperating users cannot create fresh tokens, beyond the ones issued by the issuing server. This property corresponds to “One-More-Token security” in Privacy Pass.

Theorem 1 (Unforgeability of AT_{GVRF}). Let AT_{GVRF} as defined above and with GVRF GVRF . If GVRF has group-bounded provability, then AT_{GVRF} is unforgeable.

Proof: We explain how a token forger $\mathcal{A}$ against AT_{GVRF} yields a group-bounded provability attacker $\mathcal{B}$. The reduction is straightforward and essentially only relays values.

$\mathcal{B}$ obtains pp and pk_G from the group-bounded provability challenger and forwards these to the unforgeability attacker $\mathcal{A}$. When $\mathcal{A}$ makes use of his *Generate* oracle on a public key pk , $\mathcal{B}$ needs to reply back with the corresponding certificate crt , which he obtains from his *Join* oracle $\mathcal{O}_{\text{Join}}$ on input pk . Let ctr denote the number of *Generate* queries by $\mathcal{A}$. Finally, $\mathcal{A}$ outputs $\mathcal{P} := \{p_1, \dots, p_n\}$, $t_1 := (x_1, y_1), \dots, t_l := (x_l, y_l)$ and proofs $\pi_1, \dots, \pi_\ell$.

$\text{Exp}_{\text{AT}, \mathcal{A}}^{\text{unlinkable}}(1^\lambda):$ $pp \leftarrow \text{Setup}(1^\lambda)$ $pk_S \leftarrow \mathcal{A}(pp)$ $\text{pre}_0 \leftarrow \text{AT.Generate}(U_0 : pk_S, \mathcal{A} : \perp)$ $\text{pre}_1 \leftarrow \text{AT.Generate}(U_1 : pk_S, \mathcal{A} : \perp)$ $\mathcal{Q} := \emptyset, \mathcal{Q}_{\text{open}} := \emptyset$ $b \xleftarrow{\$} \{0, 1\}$ $b^* \leftarrow \mathcal{A}^{\mathcal{O}_{\text{TokChall}}^b(\cdot), \mathcal{O}_{\text{Tok}}^0(\cdot), \mathcal{O}_{\text{Tok}}^1(\cdot)}()$ return $(b == b^*) \wedge (\mathcal{Q} \cap \mathcal{Q}_{\text{open}} == \emptyset)$	$\mathcal{O}_{\text{TokChall}}^b(p, \text{aux}):$ $\mathcal{Q} := \mathcal{Q} \cup \{p\}$ $(t, \pi) \leftarrow \text{AT.Expand}(U_b : p, \text{aux}, \text{pre}_b)$ return (t, π) $\mathcal{O}_{\text{Tok}}^0(p, \text{aux}):$ $\mathcal{Q}_{\text{open}} := \mathcal{Q}_{\text{open}} \cup \{p\}$ $(t, \pi) \leftarrow \text{AT.Expand}(U_0 : p, \text{aux}, \text{pre}_0)$ return (t, π) $\mathcal{O}_{\text{Tok}}^1(p, \text{aux}):$ $\mathcal{Q}_{\text{open}} := \mathcal{Q}_{\text{open}} \cup \{p\}$ $(t, \pi) \leftarrow \text{AT.Expand}(U_1 : p, \text{aux}, \text{pre}_1)$ return (t, π)
---	---

Figure 5: Unlinkable experiment for anonymous token schemes.

Assuming $\mathcal{A}$'s output constitutes a valid forgery, we have $\text{ctr} \cdot n < \ell$ and $t_i \neq t_j$ for all $i \neq j$ and $\text{Ver}(V : t_i, \perp, \pi_i, \mathcal{P}, sk_s) = 1$ for all $i \in [\ell]$. Thus, by the pigeonhole principle there must exist a $p^* \in \mathcal{P}$ and subset $J \subseteq [\ell]$ with $|J| > \text{ctr}$ and $x_j = p^*$ for all $j \in J$. $\mathcal{B}$ then submits $(p^*, (y_j)_{j \in J}, (\pi_j)_{j \in J})$ as forgery.

Since $(x_i, y_i) \neq (x_j, y_j)$ for all $i \neq j$ and $x_i = x_j = p^* \forall i, j \in J$, it must hold $y_i \neq y_j$ for all $i, j \in J$. Further, $\text{Ver}(V : t_i, \text{aux}_i, \pi_i, \mathcal{P}, sk_s) = 1$ for all $i \in [\ell]$ implies $\text{Ver}(pk_S, p^*, y_i, \pi_i) = 1$ for all $i \in J$. Hence, if $\mathcal{A}$ outputs a valid forgery, then $\mathcal{B}$ wins the group-bounded provability experiment. $\square$

Unlinkability. We require that it is hard to link a token to a particular user. In our abstraction of a pbATS, a user is an owner of a pre-token, and hence we demand that it is hard to decide whether two tokens, for policy elements $p \neq p'$, were expanded from the same pre-token, or different ones. We require unlinkability to hold even against a *malicious* issuer of pre-tokens. More formally:

Definition 10 (Unlinkability pbATS). We call a pbATS $\text{AT} = (\text{Setup}, \text{ServerSetup}, \text{VerSSetup}, \text{UpdatePolicy}, \text{Generate}, \text{Expand}, \text{Verify})$ *unlinkable*, if for all admissible PPT adversaries $\mathcal{A}$ there exists a negligible function $\text{negl} : \mathbb{N} \rightarrow \mathbb{R}_{>0}$, such that for all $\lambda \in \mathbb{N}$ it holds

$$|\Pr[\text{Exp}_{\text{AT}, \mathcal{A}}^{\text{unlinkable}}(1^\lambda) = 1] - \frac{1}{2}| \leq \text{negl}(\lambda),$$

where $\text{Exp}_{\text{AT}, \mathcal{A}}^{\text{unlinkable}}(1^\lambda)$ is as defined in Figure 5 and an adversary is admissible if it outputs pk_S with $\text{VerSSetup}(pk_S) = 1$ and $\text{pre}_i \neq \perp$ for $i \in \{0, 1\}$.

Theorem 2 (Unlinkability of AT_{GVRF}). Let AT_{GVRF} as defined above with GVRF. If GVRF is fully unlinkable (cf. Def. 6), then AT_{GVRF} is unlinkable.

Proof: We explain how an unlinkability adversary $\mathcal{A}$ against AT_{GVRF} yields an attacker $\mathcal{B}$ against the full unlinkability of GVRF. The reduction is straightforward since $\mathcal{B}$ essentially only forwards values between the unlinkability challenger and $\mathcal{A}$. More detailed, $\mathcal{B}$ obtains public parameters pp and runs $\mathcal{A}$ to obtain pk_S . Upon obtaining pk_0, pk_1 from the challenger, $\mathcal{B}$ sends pk_0, pk_1 to $\mathcal{A}$ during the two Generate executions, and receives back certificates $\text{crt}_0^*, \text{crt}_1^*$, which it checks and passes to the challenger. The token generation oracle $\mathcal{O}_{\text{Tok}}^i(\cdot)$ is implemented using oracle $\mathcal{O}_{\text{EvalO}}^1(\cdot)$. Finally, $\mathcal{A}$'s challenge oracle $\mathcal{O}_{\text{TokChall}}^b(p)$ is implemented using $\mathcal{B}$'s oracle

$\mathcal{O}_{\text{Eval}}^b(\cdot)$, and $\mathcal{A}$'s decision bit is adopted by $\mathcal{B}$. Clearly, $\mathcal{B}$ is successful if and only if $\mathcal{A}$ is. $\square$

5. Building Blocks

In this section we recall the definition of signatures on equivalence classes, and introduce the one-more bilinear DDH/DDHI assumption, on which our construction relies.

5.1. Signatures on Equivalence Classes

For our construction we require a randomizable signature scheme. To this end, we recall the definition of [20] of signatures on equivalence classes in the following. Let $\mathbb{G}_1$ be a group of prime order p , and $\mathbb{G}_1^* := \mathbb{G}_1 \setminus \{1_{\mathbb{G}_1}\}$. For vectors $\vec{u} \in \mathbb{G}_1^\ell$, where $\ell \in \mathbb{N}$, and $\rho \in \mathbb{Z}_p$ we denote by $\vec{u}^\rho$ the pointwise exponentiation $(u_1^\rho, \dots, u_\ell^\rho)$. Then, the equivalence relation we consider in this paper is of the form

$$\mathcal{R}_{\text{DDH}} := \{(\vec{u}, \vec{v}) \in (\mathbb{G}_1^*)^\ell \times (\mathbb{G}_1^*)^\ell \mid \exists \rho \in \mathbb{Z}_p^* : \vec{v} = \vec{u}^\rho\},$$

where in this paper we always have $\ell = 2$. For $\vec{u} \in (\mathbb{G}_1^*)^\ell$, this relation defines the equivalence class

$$[\vec{u}]_{\mathcal{R}_{\text{DDH}}} := \{\vec{v} \in (\mathbb{G}_1^*)^\ell \mid \exists \rho \in \mathbb{Z}_p^* : \vec{v} = \vec{u}^\rho\}.$$

In the following we will typically consider $\ell = 2$.

We recall the definition of signatures on equivalence classes of [20]. Note that in [20] such signatures are referred to as *structure-preserving signatures*, but as the definition does not explicitly required the signature to be structure-preserving, we will simply refer to such signatures as *signatures on equivalence classes* in the following. Further, we will fix ℓ in advance (rather than giving it as parameter to BGGen), since in the paper we will only consider $\ell = 2$.

Definition 11 (Signatures on Equivalence Classes ([20], Def. 15)). An ℓ -dimensional EQ- $\mathcal{R}$ signature scheme consists of a tuple of algorithms $\text{SIG} = (\text{BGGen}_{\mathcal{R}}, \text{KG}_{\mathcal{R}}, \text{Sign}_{\mathcal{R}}, \text{ChRep}_{\mathcal{R}}, \text{Vfy}_{\mathcal{R}})$ such that the following holds:

$\text{BGGen}_{\mathcal{R}}(1^\lambda)$: Is a probabilistic polynomial-time algorithm that on input of the security parameter 1^λ outputs $\text{BG} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, e)$, where $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T$ are groups of prime order p , g_1 is a generator of $\mathbb{G}_1$, g_2 is a generator of $\mathbb{G}_2$, and $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is a non-degenerate bilinear map.

$\text{KG}_{\mathcal{R}}(\text{BG})$: Is a probabilistic algorithm, which on input of a bilinear group BG and dimension ℓ outputs a key pair (pk, sk) .

$\text{Sign}_{\mathcal{R}}(sk, \vec{u})$: Is a probabilistic polynomial time algorithm, which on input of a secret key sk and a representative $\vec{u} \in (\mathbb{G}_1^*)^\ell$, outputs a signature σ .

$\text{ChRep}_{\mathcal{R}}(pk, \vec{u}, \sigma, \rho)$: Is a probabilistic algorithm, which on input of a public key pk , representative $\vec{u} \in (\mathbb{G}_1^*)^\ell$, signature σ and scalar $\rho \in \mathbb{Z}_p^*$, computes a new representative $\vec{v} = \vec{u}^\rho$ and an updated signature $\hat{\sigma}$, and outputs $(\vec{v}, \hat{\sigma})$.

$\text{Vfy}_{\mathcal{R}}(pk, \vec{u}, \sigma)$: Is a deterministic algorithm, which on input of a public key pk , representative $\vec{u}$ and signature σ outputs a bit $b \in \{0, 1\}$.

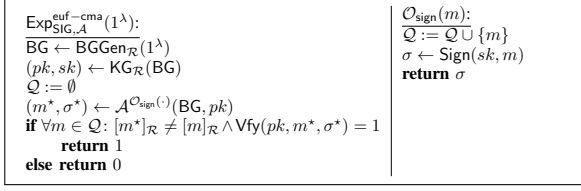


Figure 6: Unforgeability experiment for signatures on equivalence classes.

$\text{VKey}_{\mathcal{R}}(pk, sk)$: Is a deterministic algorithm, which on input of a public key pk and secret key sk outputs a bit $b \in \{0, 1\}$.

We further require the scheme to satisfy *correctness*, *EUFCMA security* and *perfect signature adaptation* as defined in the following.

Definition 12 (Correctness). We say an ℓ -dimensional EQ- $\mathcal{R}$ signature scheme $\text{SIG} = (\text{BGen}_{\mathcal{R}}, \text{KG}_{\mathcal{R}}, \text{Sign}_{\mathcal{R}}, \text{ChRep}_{\mathcal{R}}, \text{Vfy}_{\mathcal{R}})$ is *correct*, if for all security parameters $\lambda \in \mathbb{N}$, for all bilinear groups BG in the image of $\text{BGen}_{\mathcal{R}}(1^\lambda)$, for all key pairs (pk, sk) in the image of $\text{KG}_{\mathcal{R}}(\text{BG})$, for all $\vec{u} \in (\mathbb{G}^*)^\ell$, for all σ in the image of $\text{Sign}_{\mathcal{R}}(sk, \vec{u})$, it holds

$$\text{VKey}_{\mathcal{R}}(pk, sk) = 1 \text{ and } \text{Vfy}_{\mathcal{R}}(pk, \vec{u}, \sigma) = 1.$$

Further, for all $\rho \in \mathbb{Z}_p^*$, we require

$$\text{Vfy}_{\mathcal{R}}(pk, \vec{v}, \hat{\sigma}) = 1,$$

where $(\vec{v}, \hat{\sigma}) \leftarrow \text{ChRep}_{\mathcal{R}}(pk, \vec{u}, \sigma, \rho)$ and $\vec{v} = \vec{u}^\rho$.

Unforgeability for EQ- $\mathcal{R}$ signature schemes demands that it be hard to forge a verifying signature on an adversarially-chosen message m^* from a *fresh* equivalence class. Note that it is easy to produce fresh signatures using the $\text{ChRep}_{\mathcal{R}}$ algorithm on a signature obtained from the signing oracle. However, $\text{ChRep}_{\mathcal{R}}$ is “class preserving”, meaning that it can only produce signatures for messages in the same equivalence class as the original message. Hence, in the EUF-CMA experiment, the adversary is challenged to produce a forgery for a message from an equivalence class for which it has never received a signature from its oracle. We will use EUF-CMA security in our construction to allow the group manager to control the size of participant set (i.e., key holders) in a group.

Definition 13 (EUF-CMA security). We say an EQ- $\mathcal{R}$ signature scheme $\text{SIG} = (\text{BGen}_{\mathcal{R}}, \text{KG}_{\mathcal{R}}, \text{Sign}_{\mathcal{R}}, \text{ChRep}_{\mathcal{R}}, \text{Vfy}_{\mathcal{R}})$ satisfies *unforgeability*, if for all PPT adversaries $\mathcal{A}$ there exists a negligible function $\text{negl}: \mathbb{N} \rightarrow \mathbb{R}_{>0}$, such that for all $\lambda \in \mathbb{N}$

$$\Pr[\text{Exp}_{\text{SIG}, \mathcal{A}}^{\text{euf-cma}}(1^\lambda) = 1] \leq \text{negl}(\lambda),$$

where $\text{Exp}_{\text{SIG}, \mathcal{A}}^{\text{euf-cma}}$ is as defined in Figure 6.

To prove anonymity we further need that freshly generated signatures are indistinguishable from re-randomized signatures, which is captured by the notion of *perfect signature adaptation*.

Definition 14 (Perfect signature adaptation). Let $\ell > 1$. We say an ℓ -dimensional EQ- $\mathcal{R}$ signature scheme $\text{SIG} = (\text{BGen}_{\mathcal{R}}, \text{KG}_{\mathcal{R}}, \text{Sign}_{\mathcal{R}}, \text{ChRep}_{\mathcal{R}}, \text{Vfy}_{\mathcal{R}}, \text{VKey}_{\mathcal{R}})$ *perfectly adapts signatures*, if for all

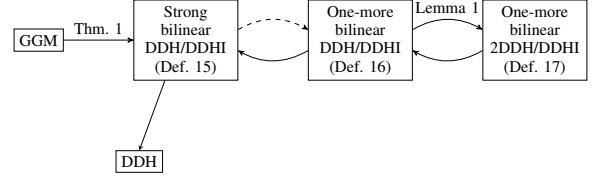


Figure 7: Implications of the various assumptions defined in this work. Dashed arrows hold only if the challenge space of the implied assumption is restricted to polynomial size.

$\lambda \in \mathbb{N}$, all bilinear groups BG in the image of $\text{BGen}_{\mathcal{R}}(1^\lambda)$ and all tuples $(sk, pk, \vec{u}, \sigma, \mu)$ with

$$\text{VKey}(pk, sk) = 1, \vec{u} \in (\mathbb{G}_1^*)^\ell, \text{Vfy}_{\mathcal{R}}(pk, \vec{u}, \sigma) = 1, \mu \in \mathbb{Z}_p^*$$

the outputs of $\text{ChRep}_{\mathcal{R}}(pk, \vec{u}, \sigma, \mu)$ and $\text{Sign}_{\mathcal{R}}(sk, \mu \cdot \vec{u})$ are identically distributed.

Instantiating Signatures on Equivalence Classes. Fuchsbauer et al. [20] give an instantiation of signature schemes on equivalence classes (Scheme 1, [20]) in the generic group model, where public keys consist of two group elements over $\mathbb{G}_2$, and signatures consist of two group elements over $\mathbb{G}_1$ plus one group element over $\mathbb{G}_2$. Computing the re-randomization of a signature requires three group exponentiation (+ two group exponentiations to compute the re-randomized message), and verifications costs five pairing evaluations.

5.2. The One-More Bilinear DDH/DDHI Assumption

For proving anonymity of our construction, we rely on a “one-more” type assumption, which can be viewed as a strengthening of bilinear variants of both the DDH and SDDHI assumption [11], [19] (although it seems incomparable to the latter). We start with a non-interactive version of this assumption, where we require the challenge space $\mathcal{X} \subset \mathbb{Z}_p$ to be of polynomial-size. We refer to this as the strong bilinear DDH/DDHI assumption. In Appendix F, we show that this assumption holds in the generic group model.

We refer the reader to Figure 7 for an overview of the cryptographic assumptions presented in this subsection, and the relations between them.

Definition 15 (Strong Bilinear DDH/DDHI Assumption).

Let $\mathcal{G}$ be a asymmetric bilinear group generator returning groups $\text{BG} := (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_3, p, g_1, g_2, e)$. Let $\mathcal{X} \subset \mathbb{Z}_p$ be a polynomial-sized set. Then, we say that the *strong bilinear DDH/DDHI assumption with respect to challenge space $\mathcal{X}$* is hard relative to $\mathcal{G}$, if for every PPT adversary $\mathcal{A}$ and every polynomial q there exists a negligible function $\text{negl}: \mathbb{N} \rightarrow \mathbb{R}_{>0}$, such that for all $\lambda \in \mathbb{N}$ it holds

$$\Pr \left[\mathcal{A} \left(\text{BG}, \{ (x, g_1^{\tau_{x,i}}, g_1^{\alpha \tau_{x,i}}, g_2^{\frac{1}{\tau_{x,i}(\alpha+x)}}) \}_{x \neq x^*, i \in [q]}, g_1^\alpha, z_b \right) = b \right] \leq \frac{1}{2} + \text{negl}(\lambda), \text{ where}$$

$$\begin{aligned} \text{BG} &\leftarrow \mathcal{G}(1^\lambda), b \xleftarrow{\$} \{0, 1\}, \alpha, \tau, y \xleftarrow{\$} \mathbb{Z}_p, x^* \xleftarrow{\$} \mathcal{X}, \\ &\forall x \in \mathcal{X} \setminus \{x^*\}, i \in [q]: \tau_{x,i} \xleftarrow{\$} \mathbb{Z}_p, \\ z_0 &:= (g_1^\tau, g_1^{\alpha\tau}, g_2^{\frac{1}{\tau(\alpha+x^*)}}), z_1 := (g_1^\tau, g_1^y, g_2^{\frac{1}{y+\tau \cdot x^*}}) \end{aligned}$$

Theorem 1 (Hardness of Strong Bilinear DDH/DDHI in GGM). Let $\mathcal{P}$ be the strong bilinear DDH/DDHI problem from Definition 15. Then any generic group algorithm sending at most t queries to the generic group oracle can solve $\mathcal{P}$ with advantage at most $\epsilon \leq (2560(|\mathcal{X}|q + 1)^5(t \log(p) + t + 1)^2)/p$.

For a proof of this theorem we refer to Appendix F.

Relation to other assumption. Our assumption is a combination of the bilinear decisional Diffie-Hellman (BDDH) assumption over $\mathbb{G}_1$, stating that given g_1^α, g_1^τ , it is hard to distinguish $g_1^{\alpha\tau}$ from random, and a bilinear version of the SDDHI assumption [11], [19] (in the following referred to as BDDHI), where the adversary has to distinguish $e(g_1, g_2)^{\frac{1}{\alpha+x^*}}$ from random and gets oracle access to an oracle $\mathcal{O}_{\text{proof}}$, which outputs $g_2^{\frac{1}{\alpha+x^*}}$ for any $x^* \neq x$. Intuitively, the only thing an adversary can do to break the new assumption is to pair elements and check for equality. To rule out such attacks, we slightly adapt the assumptions and show that the resulting combined assumption is hard in the GGM. In particular, we “weaken” the oracle of the BDDHI security assumption, as the oracle would allow an adversary to trivially break BDDH. In this sense, the assumption is incomparable to the BDDHI security assumption, but in spirit our assumption can be viewed as a strengthening of both.

One-more bilinear DDH/DDHI assumption. To remove the requirement of $\mathcal{X}$ being polynomial-sized, we introduce an interactive version of the assumption, where the adversary gets to choose the challenge x^* after seeing g_1^α , which we refer to as one-more bilinear DDH/DDHI. It is easy to see that this is implied by the non-interactive variant when the challenge space is restricted to polynomial-sized $\mathcal{X} \subset \mathbb{Z}_p$, since the reduction can simply guess the challenge x^* ahead of time, and implement $\mathcal{O}_{\text{proof}}$ using $\{(x, g_1^{\tau x}, g_1^{\alpha\tau x}, g_2^{\frac{1}{\tau x(\alpha+x^*)}})\}_{x \neq x^*}$.

Definition 16 (One-More Bilinear DDH/DDHI Assumption). Let $\mathcal{G}$ be a asymmetric bilinear group generator returning groups $\text{BG} := (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, e)$. Then, we say that the *one-more DDH/DDHI assumption* is hard relative to $\mathcal{G}$, if for every admissible PPT adversary $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$ there exists a negligible function $\text{negl}: \mathbb{N} \rightarrow \mathbb{R}_{>0}$, such that for all $\lambda \in \mathbb{N}$ it holds

$$\Pr \left[\mathcal{A}_1^{\mathcal{O}_{\text{proof}}(\cdot)}(\text{state}, z_b) = b \right] \leq \frac{1}{2} + \text{negl}(\lambda), \text{ where}$$

$$\begin{aligned} \text{BG} &\leftarrow \mathcal{G}(1^\lambda), b \xleftarrow{\$} \{0, 1\}, \alpha, \tau, y \xleftarrow{\$} \mathbb{Z}_p, Q := \emptyset, \\ (\text{state}, x^*) &\leftarrow \mathcal{A}_0^{\mathcal{O}_{\text{proof}}(\cdot)}(\text{BG}, g_1^\alpha) \\ z_0 &:= (g_1^\tau, g_1^{\alpha\tau}, g_2^{\frac{1}{\tau(\alpha+x^*)}}), z_1 := (g_1^\tau, g_1^y, g_2^{\frac{1}{y+\tau \cdot x^*}}) \text{ and} \end{aligned}$$

where $\mathcal{O}_{\text{proof}}(x)$ on the i -th query samples $\tau_i \leftarrow \mathbb{Z}_p$, outputs $(g_1^{\tau_i}, g_1^{\alpha\tau_i}, g_2^{\frac{1}{\tau_i(\alpha+x^*)}})$ and sets $Q := Q \cup \{x\}$. We say a PPT adversary $(\mathcal{A}_0, \mathcal{A}_1)$ is admissible if $x^* \notin Q$.

To show anonymity of our construction we rely on a variant of the above assumption, which allows to simulate proofs relative to two secret keys α and $\tilde{\alpha}$. We state the

assumption and show that it is implied by the one-more bilinear DDH/DDHI assumption in the Appendix G.

Definition 17 (One-More Bilinear 2-DDH/DDHI Assumption). Let $\mathcal{G}$ be a asymmetric bilinear group generator returning groups $\text{BG} := (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, g_1, g_2, e)$. Then, we say that the *one-more 2-DDH/DDHI assumption* is hard relative to $\mathcal{G}$, if for every admissible PPT adversary $\mathcal{A} = (\mathcal{A}_0, \mathcal{A}_1)$ there exists a negligible function $\text{negl}: \mathbb{N} \rightarrow \mathbb{R}_{>0}$, such that for all $\lambda \in \mathbb{N}$ it holds

$$\Pr \left[\mathcal{A}_1^{\mathcal{O}_{\text{proof}}(\cdot), \mathcal{O}_{\text{proof}}^1(\cdot)}(\text{state}, z_b) = b \right] \leq \frac{1}{2} + \text{negl}(\lambda), \text{ where}$$

$$\begin{aligned} \text{BG} &\leftarrow \mathcal{G}(1^\lambda), b \xleftarrow{\$} \{0, 1\}, \alpha_0, \alpha_1, \tau, y \xleftarrow{\$} \mathbb{Z}_p, Q := \emptyset, \\ (\text{state}, x^*) &\leftarrow \mathcal{A}_0^{\mathcal{O}_{\text{proof}}(\cdot), \mathcal{O}_{\text{proof}}^1(\cdot)}(\text{BG}, g_1^{\alpha_0}, g_1^{\alpha_1}) \\ z_b &:= (g_1^\tau, g_1^{\alpha_b\tau}, g_2^{\frac{1}{\tau(\alpha_b+x^*)}}), \end{aligned}$$

and where for $\beta \in \{0, 1\}$ $\mathcal{O}_{\text{proof}}^\beta(x)$ on the i -th query samples

$\tau_{i,\beta} \leftarrow \mathbb{Z}_p$ and outputs $\{(g_1^{\tau_{i,\beta}}, g_1^{\alpha_\beta\tau_{i,\beta}}, g_2^{\frac{1}{\tau_{i,\beta}(\alpha_\beta+x^*)}})\}$ and sets $Q := Q \cup \{x\}$. We say a PPT adversary is admissible, if it outputs $x^* \notin Q$.

Lemma 1. If the one-more bilinear DDH/DDHI assumption (Def. 16) is hard relative to $\mathcal{G}$, then so is the one-more bilinear 2-DDH/DDHI assumption (Def. 17).

6. Construction

We propose a GVRF based on the VRF by Dodis and Yampolskiy [16], adapted to asymmetric bilinear group with pairing $e: \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_3$, combined with a suitable signature scheme over equivalence classes. The construction is given in Figure 8. The main idea is as follows. A GVRF evaluation on input x is essentially a VRF evaluation of the Dodis-Yampolskiy VRF, i.e., $y := e(g_1, g_2)^{1/(x+sk)}$ is an evaluation of x under VRF public key $pk := g_1^{sk}$, and $\pi' := g_2^{1/(x+sk)}$ is the proof of correct evaluation. Using the latter one can verify correctness of y with respect to pk by means of the equations (1) $e(g_1^x \cdot pk, \pi') = e(g_1, g_2)$ and (2) $e(g_1, \pi') = y$. To enforce membership in a group, a group manager could sign pk , and evaluations are only deemed valid if they (a) verify w.r.t π' and (b) a valid signature on pk is presented. There is one remaining problem with this approach: if a user would now produce evaluations of multiple inputs, they would become linkable through pk . We prevent this by a conceptually simple measure: we let users *randomize* their public keys and the group manager’s signature on their public key, such that they become unlinkable to computationally bounded attackers (the randomization term is τ in Figure 8). This step requires us to let the group manager sign using a signature scheme on equivalence classes, where each equivalence class then corresponds to the public keys of one user. However, due the randomization of pk , also the DY PRF verification equations need to be adapted. We extend the public key by an additional component g_1 (keeping track of the randomization) and also adapt the VRF proof π' in order to get rid of the randomization factor τ in the VRF verification equations. This adaption is not generic but specific to the instance of the VRF [16] and the EQ- $\mathcal{R}$ signature [20] we consider.

Setup (1^λ): $BG \leftarrow \text{SIG.BGGen}_{\mathcal{R}}(1^\lambda)$ $BG := (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_3, p, g, e)$ $\text{crs} \leftarrow \text{PS.PGen}(BG)$ $\mathcal{X} := \mathbb{Z}_p$ $\mathcal{Y} := \mathbb{G}_3$ Output $pp := (BG, \text{crs}, \mathcal{X}, \mathcal{Y})$	VerCert (pk_G, pk, crt): Parse $pk_G := (pk^{\text{SIG}}, \pi_G)$ $b \leftarrow \text{SIG.Vfy}_{\mathcal{R}}(pk_G, pk, \text{crt})$ Output b
GroupKG (pp): $(pk^{\text{SIG}}, sk^{\text{SIG}}) \leftarrow \text{SIG.KG}(BG)$ $\pi_G \leftarrow \text{PS.PPrv}(\text{crs}, pk^{\text{SIG}}, sk^{\text{SIG}})$ $pk_G := (pk^{\text{SIG}}, \pi_G)$ $sk_G := sk^{\text{SIG}}$ Output (pk_G, sk_G)	Eval ($pk_G, pk, sk, \text{crt}, x$): Parse $pk_G := (pk^{\text{SIG}}, \pi_G)$ $\tau \xleftarrow{\$} \mathbb{Z}_p^*$ $y := e(g_1, g_2)^{1/(x+sk)}$ $\pi' := g_2^{1/(\tau(x+sk))}$ $(pk, \text{crt}) \leftarrow \text{SIG.ChRep}_{\mathcal{R}}(pk^{\text{SIG}}, pk, \text{crt}, \tau)$ Output $(y, (\pi', \tilde{pk}, \tilde{\text{crt}}), \tau)$
VerGroup (pk_G): Parse $pk_G := (pk^{\text{SIG}}, \pi_G)$ $b \leftarrow \text{PS.Ver}(\text{crs}, pk^{\text{SIG}}, \pi_G)$ Output b	Ver (pk_G, x, y, π): Parse $pk_G := (pk^{\text{SIG}}, \pi_G)$ Parse $\pi := (\pi', \tilde{pk}, \tilde{\text{crt}})$ Parse $\tilde{pk} := (pk_1, pk_2)$ If $\text{SIG.Vfy}_{\mathcal{R}}(pk^{\text{SIG}}, \tilde{pk}, \tilde{\text{crt}}) = 1 \wedge$ $e((pk_1)^x \cdot pk_2, \pi') = e(g_1, g_2) \wedge e(\tilde{pk}_1, \pi') = y$ output 1 Else output 0
KG (pk_G): $sk \xleftarrow{\$} \mathbb{Z}_p$ $pk_1 := g_1, pk_2 := g_1^{sk}$ Output $((pk_1, pk_2), sk)$	Judge ($pk_G, pk, x, y, \pi, \tau$): Parse $\pi := (\pi', \tilde{pk}, \tilde{\text{crt}})$ If $\tilde{pk} = pk^\tau \wedge \text{Ver}(pk_G, pk, x, y, \pi) = 1$ output 1 Else output 0
Join (sk_G, pk): $\text{crt} \leftarrow \text{SIG.Sign}_{\mathcal{R}}(sk_G, pk)$ Output crt	

Figure 8: Construction $\text{GVRF} = (\text{Setup}, \text{GroupKG}, \text{VerGroup}, \text{KG}, \text{Join}, \text{VerCert}, \text{Eval}, \text{Ver}, \text{Judge})$ based on the Dodis-Yampolskiy VRF [16]. Recall that all algorithms are assumed to have access to $pp = (BG, \text{crs}, \mathcal{X}, \mathcal{Y})$.

Note that to ensure an honest setup of the group manager’s signature key, we make use of a non-interactive zero-knowledge proof of knowledge PS proving that $(pk^{\text{SIG}}, sk^{\text{SIG}})$ are of the form as specified by $\text{SIG.VKey}_{\mathcal{R}}$. This proof π_G is generated in GroupKG and verified in VerGroup . That means, it only needs to be computed once by the group manager when it generates a new key pair, and it only needs to be verified once by a user for this new key pair (or this job can even be outsourced to an entity users trust). Thus, the efficiency of PS is not that crucial (unless pk_G is very short-lived). The verification of the group key setup is essential in untrusted environments involving a potentially malicious group manager as in our pseudorandomness and unlinkability experiments. More precisely, from a security proof perspective, PS is needed as the reduction will need sk^{SIG} for generating fresh signatures, which is obtained by using the extractability of PS.

We achieve the following security result for our GVRF, proven formally in Appendix E.

Theorem 3. Let $\mathcal{G}$ be a bilinear group generator, such that the one-more bilinear DDH/DDHI assumption (Def. 16) is hard relative to $\mathcal{G}$. Let $\mathcal{R} := \mathcal{R}_{\text{DDH}}$ be the DDH relation over $\mathbb{G}_1^2$ and let $\text{SIG} = (\text{BGGen}_{\mathcal{R}}, \text{KG}_{\mathcal{R}}, \text{Sign}_{\mathcal{R}}, \text{ChRep}_{\mathcal{R}}, \text{Vfy}_{\mathcal{R}}, \text{VKey}_{\mathcal{R}})$ be a 2-dimensional EQ- $\mathcal{R}_{\text{DDH}}$ signature scheme that satisfies perfect signature adaptation. Further, let $\text{PS} = (\text{PGen}, \text{PTGen}, \text{PPrv}, \text{PVer}, \text{PSim}, \text{PExt})$ be a non-interactive zero knowledge proof of knowledge for the relation defined by $\text{SIG.VKey}_{\mathcal{R}}$. Then, the GVRF given in Figure 8 is a group VRF with unlinkability. Further, if SIG additionally satisfies EUF-CMA secu-

rity (and the proof system PS satisfies zero knowledge as before), the GVRF also satisfies group-bounded provability. Finally, the GVRF satisfies unique opening unconditionally.

7. Implementation

Set-up. By directly comparing the execution times of the pbATS instances of Privacy Pass and GVRF, we evaluate the practical feasibility of our proposed scheme. You can find our code on GitHub here ⁵. For this purpose, we implemented both schemes as a multi-threaded application in C++23 and are only inspecting the pure cryptography or payload sizes. The client is running on a notebook equipped with an AMD® Ryzen® 7 PRO 5875U@ 4.50 GHz, 32 GB RAM and running Ubuntu 22.04.1 LTS (kernel version 6.5), whereas the servers are running on an AMD EPYC 9274F 24-Core Processor @ max. 4.30 GHz with 128 GB RAM and Ubuntu 24.04.2 LTS (kernel version 6.8.0). Server-side utilizes all CPU cores for optimal performance, while client-side is not abusing many threads to reflect a device with low-resources (e.g. Smartphone). We also use the RELIC Toolkit v.0.7.0 [2] implementation to instantiate GVRFs over the pairing-friendly BLS12-381 curve and Privacy Pass over the NIST P-256 curve. Both curves provide approximately 128 bits of security and are securitywise comparable to each other. Further, we use the aforementioned scheme in [20] to create signatures on equivalence classes for GVRF’s. To the best of our knowledge, there is currently no working and fast arithmetic backend available from RELIC for the x86 architecture for P-256. For fairness reasons, we therefore compare both constructions without optimizations but include the benchmarks for GVRF with optimizations in the column “GVRF*” of the Tables.

Token Generation. Privacy Pass clearly has the overall performance advantage when examining the entire token generation process on both the client and server side in Table 1, but places a heavier load on the issuing servers in terms of computation and bandwidth. For example, the entire token generation of 25 tokens takes 18.33 ms for Privacy Pass, compared to 85.24 ms for GVRF and 47.76 ms for GVRF*. Doubling the number of tokens roughly doubles the execution time in the case of Privacy Pass.

Further, the results show that server-side AT.Generate is independent of the number of tokens for GVRF and takes a constant 435 μs and for GVRF* 163 μs with a total of 307 bytes to transmit. The request consists of 2 $\mathbb{G}_1$ elements, namely the public key, which is 102 bytes, and the response is a certificate on this public key, which is just an SPS-EQ signature consisting of 2 $\mathbb{G}_1$ and 1 $\mathbb{G}_2$ element, and a global policy of 2 $\mathbb{Z}_p$ elements. In total the response is 205 bytes. GVRF outperform Privacy Pass with a 10-token batch, where GVRF* is comparable with the server-side issuance time of 2 Privacy Pass tokens. Especially in a CDN setting, where millions of requests per second are expected for the issuing servers, server-side bandwidth and computational efficiency become important factors. For token sizes greater than 1, GVRF performs better in both, through the constant communicational and computational

5. <https://github.com/dennisfInf/GVRF>

TABLE 1: "Token generation costs (AT.Generate() + AT.Expand(), where AT.Expand() can be performed offline) on client and server-side for Privacy Pass (PP), GVRF (G), and GVRF* (G*), averaged over 100k runs. Timings are in ms; throughput (issuing requests per second (issue req/s)). Network latency is excluded, as both protocols require only one communication round. Communication costs are given in bytes and inspects the payload size only. **Best performance** in each row is highlighted in **bold**. (Note that for token generation time per client, we mark both best overall time (PP) and best online time (G* for > 1 token) in **bold**.)"

No. of Tokens	Token Generation Per Entity (ms)						Communication Costs (bytes)				Throughput (issue requests/s)		
	Server			Client			Server		Client		Server		
	PP	G	G*	PP	G	G*	PP	G/G*	PP	G/G*	PP	G	G*
1	0.081	0.435	0.163	1.1 + 0.006	2.8 + 3.2	1.6 + 1.8	107	205	37	102	12300	2300	6104
10	0.838	0.435	0.163	4.1 + 0.06	2.8 + 32	1.6 + 18	404	205	334	102	1193	2300	6104
25	1.98	0.435	0.163	16.2 + 0.15	2.8 + 82	1.6 + 46	899	205	829	102	505	2300	6104
50	4.07	0.435	0.163	31 + 0.3	2.8 + 164	1.6 + 91	1724	205	1654	102	246	2300	6104
100	8.47	0.435	0.163	61 + 0.6	2.8 + 324	1.6 + 178	3374	205	3304	102	118	2300	6104

TABLE 2: "Average web-server-side verification AT.Verify() times (ms) for Privacy Pass (PP), GVRF (G) and GVRF* (G*), with communication costs and server throughput (verification requests per second). Throughput reflects cryptographic operations only. Communication costs (in bytes) inspect the payload size only. Excluding network latency. **Best performance** in each row is highlighted in **bold**."

Verify (ms)			Communication (bytes)		Throughput (verify requests/s)		
PP	G	G*	PP	G/G*	PP	G	G*
0.094	7	4	71	781	10 600	142	244

costs, since one issuing request is required for a client to generate tokens as many as the policy allows. Looking at the server-side throughput of 10 token batches in Table 1: GVRF outperforms Privacy Pass by a factor of 2 and GVRF* by a factor of almost 6 with a constant number of 2300 issuing req/s for GVRF and 6104 issuing req/s for GVRF*. However, on the client side, since expanding a token by using pairings is cryptographically expensive, it takes 3.2ms per token generation for GVRF and 1.8ms for GVRF*.

Expanding tokens from pre-tokens is done on client-side and can be done offline. The expansion of tokens can be done in the background by pre-computation and also on-demand, where an amount of 3ms is an imperceptible delay for a human. For both protocols, multiple executions of AT.Expand are independent and can happen in parallel to speed up computation. Furthermore, a signing request for 50 pre-tokens in Privacy Pass transmits 1.654 kB of data and the response 1.724 kB, increasing with the number of tokens, because elliptic curve points equal to the number of tokens must be sent in the request and response. In AT.Expand, on the other hand, Privacy Pass is able to use a keyed hash function based on SHA-256 in our benchmarks, which outperforms expensive public key operations in GVRF. The token size is 71 bytes for Privacy Pass and 781 bytes for GVRF, since it consists of multiple group elements. Thus, GVRF require 11 times more memory than Privacy Pass.

Token Verification. Overall, Privacy Pass is performing significantly better in the verification process than GVRF. For Privacy Pass it takes 0.01 ms to verify a token, for GVRF 7 ms and for GVRF* 4 ms. As the latter is way slower than Privacy Pass and puts more load on the server-

side, 7ms and 4ms are still acceptable by distributing the load across multiple verification servers. When looking at the server throughput, where Privacy Pass achieves 10 600 verification req/s, GVRF 142 verification req/s and GVRF* 244 verification req/s, GVRF-based AT needs multiple servers to keep up with Privacy Pass. Token verification requires transmitting $4 \mathbb{G}_1$, $2 \mathbb{G}_2$, $1 \mathbb{G}_T$, $1 \mathbb{Z}_p$ elements for GVRF, which are 781 bytes in our implementation. On the other hand, Privacy Pass requires transmitting $1 \mathbb{Z}_p$ element, 1 MAC (32bytes), which were 71 bytes in our benchmarks. Here, Privacy Pass clearly outperforms GVRF communication and computation-wise.

8. Deployment Considerations

In this section we discuss several aspects which should be taken into account when considering a deployment of GVRF-based pbATS as an alternative to Privacy Pass. In particular, we want to emphasize that GVRF-based pbATS achieve a different performance tradeoff than Privacy Pass: a lower computational effort for the issuing servers at the cost of a higher effort for the verification servers and clients. Also, they come with somewhat weaker unlinkability guarantees but offer the feature of updatable policies which cannot be easily simulated with Privacy Pass.

Performance tradeoff. As backed by our benchmarks in Section 7, GVRF-based schemes feature a relatively light token issuance phase for servers, and can hence avoid bottlenecks when token issuance is carried out by only few servers. However, it is important to consider the tradeoffs: GVRF clients need to invest more computing time to generate their tokens, and verification of the tokens is significantly more complex than in Privacy Pass. GVRF-based tokens are hence not suitable when token redemption is carried out by only a few servers.

Trading faster server-side generation time for slower server-side verification time becomes beneficial in an environment where a high load of issuance requests is expected to be handled by a few servers, while verification is distributed across many origins, e.g., websites. Such a scenario is also motivated by the deployment models in Sections 4.2 and 4.4, and the idea of issuer centralization in Section 5.2 of RFC 9576 [27]. Centralizing the issuer party has the advantage that the secret key only needs to be shared by a few servers, thus minimizing the risk of a key exposure. Distributing the verifier seems reasonable,

since verifiers do only need to share public information like the public key and access to a distributed or centralized database to prevent double-spending, where efficient solutions for distributed databases already exist.

GVRF-based schemes seem to be well suited for such an environment as communication and server-side costs in the token issuance process are constant and comparable to the issuance process of a single token in Privacy Pass (cf. Table 1), even though the client is able to extend its pre-token to not only one but as many tokens as the policy allows. Moreover, the number of issuing requests can further be reduced by using the policy update feature which enables clients already in possession of a valid pre-token to generate new tokens without any interaction. On the other hand, the high verification times of GVRF tokens need to be compensated by distributing the load to a high number of verifying servers if client-behavior does not already induce an appropriate distribution.

Limits of our unlinkability notion. In the unlinkability experiment we capture that an adversary cannot distinguish whether two accepted tokens (p, t, π) and (p', t', π') for policy elements $p \neq p'$ were generated by the same user or two different users. This does not prevent leakage based on the used policy elements though.

First of all, it is important that users choose the policy element at random from the set of available policy elements. Otherwise, if users use tokens in a deterministic sequence (i.e., $p_1, p_2, \dots$, according to some ordering), an adversary corrupting both the issuing server and a set of websites can potentially link queries based on the used policy, as they give an ordering on the redeemed tokens. In an extreme case, where a single user makes more requests than any other user, the latest token redemptions by this user can be fully linked based on the advanced policy elements.

Even with this mitigation, the policy elements give some leakage. Namely, if $p = p'$, our scheme reveals that two accepted tokens must stem from two different anonymous users (in the following we refer to this as “unlinking” leakage). This kind of leakage is inherent to our definition, where verifying a token requires to know the exact p , as unforgeability requires that a user cannot generate more than one accepting token relative to the same p to allow for rate limiting.

Whether the unlinking leakage is acceptable requires a careful case study depending on the setting. We believe that this kind of leakage is not significant in large-scale settings, such as Cloudflare’s with 32M requests per second, where the probability that two requests stem from different users is high in any case. To see this, consider the following simplified setting, where, say, all servers accept tokens relative to some fixed policy elements $\{p_1, \dots, p_{100}\}$, and there are $100 \cdot N$ token redemptions in total. What an adversary corrupting both the issuing server and all redemption servers can now do, is to split up all redemption queries (consisting of a token and the redemption server) into 100 buckets based on the used policy element, which is part of the token. As we assume that policies are chosen at random, we expect all buckets to roughly contain N redemption queries (which, for simplification, we assume to be exactly N in the following). Now, if we look at a single user, we know

that he can have made at most one redemption query in each bucket (assuming he only made one pretoken query), as he cannot have redeemed two tokens corresponding to the same policy element p . As he could have made 0 or 1 out of N possible redemption queries in each bucket, this gives $N + 1$ possible choices per bucket (corresponding to no redemption, redemption corresponding to redemption query 1, $\dots$, redemption corresponding to redemption query N), resulting in $(N + 1)^{100}$ possibilities in total. While this leakage can be combined with additional leakage (such as timing leakage, which can also be made in a fully unlinkable setting), we believe that the achieved unlinkability gives a high level of individual privacy whenever the bucket sizes are expected to be large. Additionally, redemption servers that receive many queries are expected to occur in all buckets evenly, therefore not allowing for any leakage.

In small-scale settings, however, this leakage can be an issue. For instance, assume the extreme case of a size-2 policy and 2-user setting with $N = 4$ websites: If both users generate tokens relative to the same p_1, p_2 to visit websites W_1, W_2 and W'_1, W'_2 , respectively. Then, an adversary corrupting both the issuing server and websites can deduce that both users visited exactly two websites and that either W_1 and W_2 or W'_1 and W'_2 were visited by the same user (and similarly for W'_1), corresponding to an entropy loss of 50%.

We note that the leakage in our construction could be avoided by p not being part of the token, but a range proof, that proves that p is within the policies interval. Since this slows down verification even further, we accepted this leakage in our construction. Privacy Pass, on the other hand, achieves a stronger notion of unlinkability, where two requests can neither be linked to the same, nor to different users, and is thus the preferable choice in small-scale or other settings where the unlinking leakage is not acceptable.

Policies and their usage (cf. Appendix D for details). Policies in our GVRF-based pbATS instantiation are public subsets $\mathcal{P} \subset \mathbb{Z}_p = \{0, \dots, p - 1\}$, where p is a large prime. As for each $x \in \mathcal{P}$, a user can generate exactly one token using its pre-token (by evaluation with x) which can only be spent once, $|\mathcal{P}|$ determines the number of tokens currently available per pre-token. $\mathcal{P}$ can be represented by an interval $[a, b] \subset \mathbb{Z}_p$, where it suffices to make a and the length of the interval known to the users.

In this way, one can easily enforce global policies, i.e., fixing a maximum number of tokens which can be used by each pre-token holder and redeemed at all websites. It also allows to realize more advanced policies like time-, website-, content-, or service-dependent policies by encoding time stamps or IDs as prefix of $\mathbb{Z}_p$ values. For instance, the policies $\mathcal{P}_{9999} = [999901, 999930]$ and $\mathcal{P}_{8888} = [888801, 888810]$ allow a pre-token holder to redeem 30 tokens at the website with ID 9999 and only 10 at the website with ID 8888, respectively. Combinations of different policy types are also possible. Note that we do not support user-specific policies as Rate-limited Privacy Pass [14] does at the cost of introducing a trusted mediator entity.

Another important feature are policy updates as a means to react to the current or expected network situation

by *retrospectively* decreasing or increasing the number of unspent tokens in circulation. For instance, if for the current policy there is an overload of token-based website requests, we can decrease the total number of remaining tokens by decreasing the size of the policy interval. In Privacy Pass, not inherently supporting the concept of updatable policies, the effects of advanced policies can only be approximated in an inefficient way, e.g., by frequently rotating pk_S (time-dependent policies) or running several instances of Privacy Pass in parallel (website-/content-/service-dependent policies). In general, with Privacy Pass we cannot really implement the main goal of policy updates, i.e., to retrospectively adjust the number of unspent tokens. All we can do is to invalidate all unspent tokens at once by renewing the issuing key. On the other hand, a potential downside of policy updates affecting *all* holders of valid pre-tokens is the following: As long as pk_S stays the same, we cannot selectively adapt only the size of *newly* issued token batches without also adjusting the size of all previous ones, which however, is easily possible in Privacy Pass.

References

- [1] Masayuki Abe, Jens Groth, Kristiyan Haralambiev, and Miyako Ohkubo. Optimal structure-preserving signatures in asymmetric bilinear groups. In Phillip Rogaway, editor, *CRYPTO 2011*, volume 6841 of *LNCS*, pages 649–666. Springer, Berlin, Heidelberg, August 2011.
- [2] D. F. Aranha and C. P. L. Gouvêa. RELIC is an Efficient Library for Cryptography. <https://github.com/relic-toolkit/relic>, 2020.
- [3] Balthazar Bauer, Georg Fuchsbauer, and Julian Loss. A classification of computational assumptions in the algebraic group model. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 121–151. Springer, Cham, August 2020.
- [4] Mira Belenkiy, Melissa Chase, Markulf Kohlweiss, and Anna Lysyanskaya. Compact e-cash and simulatable VRFs revisited. In Hovav Shacham and Brent Waters, editors, *PAIRING 2009*, volume 5671 of *LNCS*, pages 114–131. Springer, Berlin, Heidelberg, August 2009.
- [5] Fabrice Benhamouda, Tancrède Lepoint, Michele Orrù, and Mariana Raykova. Publicly verifiable anonymous tokens with private metadata bit. *IACR Cryptol. ePrint Arch.*, page 4, 2022.
- [6] Fabrice Benhamouda, Mariana Raykova, and Karn Seth. Anonymous counting tokens. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part II*, volume 14439 of *LNCS*, pages 245–278. Springer, Singapore, December 2023.
- [7] Olivier Blazy, Georg Fuchsbauer, Malika Izabachène, Amandine Jambert, Hervé Sibert, and Damien Vergnaud. Batch Groth-Sahai. In Jianying Zhou and Moti Yung, editors, *ACNS 10 International Conference on Applied Cryptography and Network Security*, volume 6123 of *LNCS*, pages 218–235. Springer, Berlin, Heidelberg, June 2010.
- [8] Jan Bobolz, Fabian Eidens, Stephan Krenn, Daniel Slamanig, and Christoph Striecks. Privacy-preserving incentive systems with highly efficient point-collection. In Hung-Min Sun, Shih-Pyng Shieh, Guofei Gu, and Giuseppe Ateniese, editors, *ASIACCS 20*, pages 319–333. ACM Press, October 2020.
- [9] Xavier Boyen. The uber-assumption family (invited talk). In Steven D. Galbraith and Kenneth G. Paterson, editors, *PAIRING 2008*, volume 5209 of *LNCS*, pages 39–56. Springer, Berlin, Heidelberg, September 2008.
- [10] Nicholas Brandt, Dennis Hofheinz, Julia Kastner, and Akin Ünäl. The price of verifiability: Lower bounds for verifiable random functions. In Eike Kiltz and Vinod Vaikuntanathan, editors, *TCC 2022, Part II*, volume 13748 of *LNCS*, pages 747–776. Springer, Cham, November 2022.
- [11] Jan Camenisch, Susan Hohenberger, Markulf Kohlweiss, Anna Lysyanskaya, and Mira Meyerovich. How to win the clonewars: Efficient periodic n-times anonymous authentication. In Ari Juels, Rebecca N. Wright, and Sabrina De Capitani di Vimercati, editors, *ACM CCS 2006*, pages 201–210. ACM Press, October / November 2006.
- [12] Jan Camenisch, Susan Hohenberger, and Anna Lysyanskaya. Compact e-cash. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 302–321. Springer, Berlin, Heidelberg, May 2005.
- [13] Melissa Chase, F. Betül Durak, and Serge Vaudenay. Anonymous tokens with stronger metadata bit hiding from algebraic MACs. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 418–449. Springer, Cham, August 2023.
- [14] Hien Chu, Khue Do, and Lucjan Hanzlik. On the security of rate-limited privacy pass. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26-30, 2023*, pages 2871–2885. ACM, 2023.
- [15] Alex Davidson, Ian Goldberg, Nick Sullivan, George Tankersley, and Filippo Valsorda. Privacy pass: Bypassing internet challenges anonymously. *PoPETs*, 2018(3):164–180, July 2018.
- [16] Yevgeniy Dodis and Aleksandr Yampolskiy. A verifiable random function with short proofs and keys. In Serge Vaudenay, editor, *PKC 2005*, volume 3386 of *LNCS*, pages 416–431. Springer, Berlin, Heidelberg, January 2005.
- [17] Alex Escala and Jens Groth. Fine-tuning Groth-Sahai proofs. In Hugo Krawczyk, editor, *PKC 2014*, volume 8383 of *LNCS*, pages 630–649. Springer, Berlin, Heidelberg, March 2014.
- [18] Valerie Fetzer, Max Hoffmann, Matthias Nagel, Andy Rupp, and Rebecca Schwerdt. P4TC - provably-secure yet practical privacy-preserving toll collection. *Proc. Priv. Enhancing Technol.*, 2020(3):62–152, 2020.
- [19] Matthew K. Franklin and Haibin Zhang. Unique group signatures. In Sara Foresti, Moti Yung, and Fabio Martinelli, editors, *ESORICS 2012*, volume 7459 of *LNCS*, pages 643–660. Springer, Berlin, Heidelberg, September 2012.
- [20] Georg Fuchsbauer, Christian Hanser, and Daniel Slamanig. Structure-preserving signatures on equivalence classes and constant-size anonymous credentials. *Journal of Cryptology*, 32(2):498–546, April 2019.
- [21] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 33–62. Springer, Cham, August 2018.
- [22] Ruti Gafni and Idan Nagar. Captcha – security affecting user experience. *Issues in Informing Science and Information Technology*, 13:63–77, 01 2016.
- [23] Chaya Ganesh, Claudio Orlandi, and Daniel Tschudi. Proof-of-stake protocols for privacy-aware blockchains. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part I*, volume 11476 of *LNCS*, pages 690–719. Springer, Cham, May 2019.
- [24] Jens Groth. Fully anonymous group signatures without random oracles. In Kaoru Kurosawa, editor, *ASIACRYPT 2007*, volume 4833 of *LNCS*, pages 164–180. Springer, Berlin, Heidelberg, December 2007.
- [25] Jens Groth and Amit Sahai. Efficient non-interactive proof systems for bilinear groups. In Nigel P. Smart, editor, *EUROCRYPT 2008*, volume 4965 of *LNCS*, pages 415–432. Springer, Berlin, Heidelberg, April 2008.
- [26] Scott Hendrickson, Jana Iyengar, Tommy Pauly, Steven Valdez, and Christopher A. Wood. Rate-Limited Token Issuance Protocol. Internet-Draft draft-ietf-privacypass-rate-limit-tokens-06, Internet Engineering Task Force, April 2024. Work in Progress.
- [27] Alex Davidson Jana Iyengar and Christopher A. Wood. The Privacy Pass Architecture. Internet-Draft draft-ietf-privacypass-architecture, Internet Engineering Task Force, June 2024. Completed.

- [28] Eike Kiltz. Chosen-ciphertext security from tag-based encryption. In Shai Halevi and Tal Rabin, editors, *TCC 2006*, volume 3876 of *LNCS*, pages 581–600. Springer, Berlin, Heidelberg, March 2006.
- [29] Ben Kreuter, Tancrède Lepoint, Michele Orrù, and Mariana Raykova. Anonymous tokens with private metadata bit. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part I*, volume 12170 of *LNCS*, pages 308–336. Springer, Cham, August 2020.
- [30] Ueli M. Maurer. Abstract models of computation in cryptography (invited paper). In Nigel P. Smart, editor, *10th IMA International Conference on Cryptography and Coding*, volume 3796 of *LNCS*, pages 1–12. Springer, Berlin, Heidelberg, December 2005.
- [31] Torben P. Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In Joan Feigenbaum, editor, *CRYPTO'91*, volume 576 of *LNCS*, pages 129–140. Springer, Berlin, Heidelberg, August 1992.
- [32] Lior Rotem and Gil Segev. Algebraic distinguishers: From discrete logarithms to decisional uber assumptions. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part III*, volume 12552 of *LNCS*, pages 366–389. Springer, Cham, November 2020.
- [33] J. T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *Journal of the Association for Computing Machinery*, 27(4):701–717, 1980.
- [34] Victor Shoup. Lower bounds for discrete logarithms and related problems. In Walter Fumy, editor, *EUROCRYPT'97*, volume 1233 of *LNCS*, pages 256–266. Springer, Berlin, Heidelberg, May 1997.
- [35] Tjerdand Silde and Martin Strand. Anonymous tokens with public metadata and applications to private contact tracing. In Ittay Eyal and Juan A. Garay, editors, *FC 2022*, volume 13411 of *LNCS*, pages 179–199. Springer, Cham, May 2022.
- [36] Sofia Celi Alex Davidson Steven Valdez and Christopher A. Wood. Privacy Pass Issuance Protocols. Internet-Draft draft-ietf-privacypass-protocol, Internet Engineering Task Force, June 2024. Completed.
- [37] Mark Zhandry. To label, or not to label (in generic groups). In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part III*, volume 13509 of *LNCS*, pages 66–96. Springer, Cham, August 2022.

Appendix A. Related Cryptographic Primitives

In this section we contextualize group VRFs among related cryptographic primitives. Ganesh et al. [23] were the first to introduce the concept of *anonymous VRFs*. Their central idea is that anonymity requires non-uniqueness of public keys, such that each evaluation w.r.t a specific PRF key can be verified under a *different* public key. Their construction for the PRF $H(x)^{sk}$ is secure in the random oracle model under the DDH assumption. However, as for standard VRFs, anybody can generate a VRF key pair and hence there is no built-in way to control the anonymity set. Ganesh et al. [23] suggest to circumvent this drawback by proving in zero-knowledge that a randomized public key was generated from a public list of keys. In our work, we suggest a new notion of anonymous VRFs which captures the anonymity set. Namely, we generalize their model to the group setting, allowing us to control the amount of PRF keys in the system through a group manager. While we follow the idea of [23] to introduce non-unique public keys, we select the DY PRF as underlying PRF and demonstrate that it can be augmented with anonymity guarantees in the standard model, without relying on generic NIZKs.

Unique group signatures are a concept that is closely related to group VRFs. A notable difference is that signatures do not need to be pseudorandom, and thus certain

	Anonymous VRFs [23]	Group VRFs this work	Unique group signatures [19]
Pseudorandomness	✓	✓	(✓)
Anonymity	✓	✓	✓
Dynamic (adaptive joining)	✓	✓	✓
Controlled anonymity set	✗	✓	✓
Anon. against group manager	-	✓	✗
Security against malicious pk	✓	✓	✓

TABLE 3: Cryptographic primitives for producing verifiable but anonymous pseudorandomness. Group VRFs are the only primitive that allow to control the anonymity set by an authority without revealing the evaluator’s identity to that authority. The “-” for [23] is because they do not require a group manager. Unique group signatures are not necessarily pseudorandom, but the dynamic construction of [19] is.

applications such as lotteries work better with VRFs than signatures. Franklin and Zhang [19] introduce unique group signatures, and build them from VRFs, committed PRF keys, and NIZK proofs of correct computation [4]. While their construction relies on similar assumptions to ours, it is significantly less efficient in terms of signature/image and proof size. Further, Franklin et al. [19] require (and allow) the group manager to deanonymize signers while we aim at avoiding such a central and powerful entity.

Finally, numerous works explicitly equip the Dodis-Yampolskiy PRF with anonymity, e.g., [18], [12], [8], through usage of generic NIZK proofs of correct computation, without revealing the public key. Our group VRF might be plugged into these constructions to obtain more efficient variants of these systems.

We provide a detailed comparison of related schemes’ properties and costs in Table 3. The table shows all cryptographic primitives in the literature that allow to produce verifiable pseudorandomness in an anonymous fashion. The crucial difference is the *control over the anonymity set*, which does not exist in anonymous VRFs as presented in [23]. Our work shows that we can add such control. While this was previously already demonstrated in [19] through the concept of unique group signature, our work provides two significant improvements: first, we demonstrate that control of the anonymity set can be added (in terms of a group manager) but *without* sacrificing the anonymity property to the group manager, as is the case in [19]. Second, we greatly improve upon the efficiency of [19], which relies on computationally heavy tools such as Groth-Sahai proofs [24] and tag-based CCA-secure encryption [28]. We further note that the dynamic construction in [19] does not come with a formal security proof.

Appendix B. Non-Interactive Zero-Knowledge Proofs

In the following we give a definition of non-interactive zero knowledge proofs of knowledge based on [25], where we additionally require a knowledge extractor. Note that the latter can be instantiated by adding an encryption of the witness and extending the proof system to the corresponding language. The trapdoor will then contain a key to decrypt.

Definition 18. Let $\mathcal{R}$ be an efficiently computable relation. A non-interactive zero-knowledge proof system consists of a tuple of PPT algorithms $\text{PS} := (\text{PGen}, \text{PTGen}, \text{PPrv}, \text{PVer}, \text{PSim}, \text{PExt})$ of the following syntax.

- $\text{PGen}(pp)$ on input of the public parameters pp generates a common reference string crs .
- $\text{PTGen}(pp)$ on input of the public parameters pp generates a common reference string crs and additionally a trapdoor td .
- $\text{PPrv}(\text{crs}, x, w)$ on input of a common reference string crs , statement x and witness w with $(x, w) \in \mathcal{R}$ outputs a proof π .
- $\text{PVer}(\text{crs}, x, \pi)$ on input of a common reference string crs , statement x and proof π outputs a bit $b \in \{0, 1\}$.
- $\text{PSim}(\text{crs}, \text{td}, x)$ on input a common reference string crs with trapdoor td and a statement x outputs a proof π .
- $\text{PExt}(\text{crs}, \text{td}, x, \pi)$ on input a common reference string crs with trapdoor td and a statement x with proof π outputs a witness w .

We further require the following to hold.

- **Completeness:** For all public parameters pp , for all $(x, w) \in \mathcal{R}$, we have $\text{PVer}(\text{crs}, x, \pi) = 1$.
- **Zero knowledge:** For all public parameters pp , we have that the distribution of $\text{crs} \leftarrow \text{PGen}(pp)$ and crs obtained via $(\text{crs}, \text{td}) \leftarrow \text{PTGen}(pp)$ are computationally indistinguishable. For all statements $x \in \mathcal{L}$ with w such that $(x, w) \in \mathcal{R}_{\mathcal{L}}$ we further have that $\text{PPrv}(\text{crs}, x, w)$ and $\text{PSim}(\text{crs}, \text{td}, x)$ are identically distributed for $(\text{crs}, \text{td}) \leftarrow \text{PTGen}(1^\lambda)$.
- **Knowledge soundness:** For all public parameters pp , for all PPT adversaries $\mathcal{A}$ for $(\text{crs}, \text{td}) \leftarrow \text{PTGen}(pp)$, for all statements x , for $\pi \leftarrow \mathcal{A}(pp, \text{crs})$ the following holds: If $\text{PVer}(\text{crs}, x, \pi) = 1$, then the extractor yields a witness $w \leftarrow \text{PExt}(\text{crs}, \text{td}, x, \pi)$ such that $(x, w) \in \mathcal{R}$, except with negligible probability. Here, the probability is taken over the random coins of $\text{PTGen}, \mathcal{A}$ and PExt .

Appendix C. GVRF from NIZK

Combining a PRF with a NIZK proof is often the approach that is taken in privacy-preserving protocols to obtain an unlinkable and verifiable PRF. Our GVRF instantiation is significantly more efficient than such a construction. Of course, the exact performance gap highly depends on the concrete approach and chosen primitives. For concreteness, let us consider an approach where the proof of correct PRF evaluation $F_{sk}(x) = y$ is for a language of the following form $\{(x, y) \mid \exists sk, C, r, \sigma \text{ s.t. } C = \text{Com}(sk; r), \text{Ver}(pk^{\text{SIG}}, \sigma, C) = 1, F_{sk}(x) = y\}$, where pk^{SIG} is the signature verification key of an authority (e.g., the group manager for GVRFs). When instantiating the above using the DY-PRF [16], Pedersen commitments [31], the optimal structure-preserving signature scheme by ABE [1], and (SXDH-based) Groth-Sahai NIZKs [25], we obtain the following estimates based on Figure 13 in [17] and Table 1 in [7]: generating a proof requires

about 90 $\mathbb{G}_1$ and 60 $\mathbb{G}_2$ exponentiations, verifying it about 120 pairing evaluations. This compares to 4 $\mathbb{G}_1$ and 2 $\mathbb{G}_2$ exponentiations for proof generation and 7 pairing evaluations plus one $\mathbb{G}_1$ exponentiation for proof verification in our GVRF construction.

Appendix D. Policies and Their Usage

Using the instantiation of a GVRF proposed in Section 6, the policies $\mathcal{P}$ of our pbATS scheme are subsets of $\mathbb{Z}_p = \{0, \dots, p-1\}$, where p is the prime order of a bilinear group setting. The size of $\mathcal{P}$ determines the number of tokens available to each user holding a pre-token valid under the current pk_S . Policies are public sets that do not need to contain random numbers, but may consist of consecutive numbers. Hence, $\mathcal{P}$ might be represented by an interval $[a, b] \subset \mathbb{Z}_p$. In this case, to fix a policy it suffices to make a and the length of the interval public (e.g., by transmitting it to the user after solving a CAPTCHA, just before token verification, etc.).

Basic policy types. Using this approach, several basic policy types can be implemented, including:

- *Global policies* fix a maximum number of tokens which can be used by each pre-token holder and redeemed at all websites for all possible contents and services until the policy gets updated. A global policy can be defined by fixing an interval $[a, b] \subset \mathbb{Z}_p$.
- *Time-dependent policies* fix a maximum number of tokens which can be used by each pre-token holder within a certain period of time. To define such a policy, the idea is to encode rough fixed-length time stamps ts (e.g., in the format YYYY-MM-DD-HH) as “prefix” of a $\mathbb{Z}_p$ value and append a fixed-length counter, i.e., $\mathcal{P}_{ts,n} := \{ts \parallel \text{pad}(c) \mid 1 \leq c \leq n\} = [ts \parallel \text{pad}(1), ts \parallel n] \subset \mathbb{Z}_p$, where $\text{pad}(c)$ is a representation of c using the same number of digits as n (i.e., zero-digits are prepended if needed). For instance, $\mathcal{P} = [202503101301, 202503101330]$ allows each pre-token holder to spend 30 tokens on March 10th, 2025, between 13:00 and 13:59. Clearly, the token verifying entities will need to automatically update the policy they accept according to the current time. If we assume an agreement on a common time zone and that the clocks of users are roughly synchronized with those of the verifying entities, it suffices to only transfer the current n to the user to fix a policy. Note that for a new policy of size n , each pre-token holder will be able to redeem n tokens independent of how many tokens it spent wrt. the previous policy.
- *Website/content/service-dependent policies* fix website-, content-, or service-specific maximum numbers of tokens which can be used by each pre-token holder. For instance, it could be useful to allow pre-token holders to redeem more tokens (i.e., make a larger number of requests without CAPTCHA) for static, small-sized content than for costly dynamically-generated (e.g., by API requests) or large-size content (e.g., software downloads). To define such policies, we can follow a similar approach as for time-dependent policies: we assign fixed-length unique IDs to websites, content or

service types, which are then encoded as “prefix” of a $\mathbb{Z}_p$ value and append by a fixed-length counter, i.e., $\mathcal{P}_{ID,n} := [ID || \text{pad}(1), ID || n]$. Note that when a pre-token holder has redeemed all its tokens associated with $\mathcal{P}_{ID_i, n_i}$, it is still able to redeem tokens associated with a different policy $\mathcal{P}_{ID_j, n_j}$ without being forced to request a new pre-token (i.e., solve a CAPTCHA).

Combination of basic policies. Obviously, it could also be useful to combine certain basic policies. For instance, we can easily obtain time-dependent policies for different content types by just prepending IDs to the definitions of the time-dependent intervals. Another interesting use-case could be combining a global policy with content-dependent policies in the sense that a pre-token owner has an overall limit of tokens it can spent as well as content-specific limits. For this purpose, we enforce that every time a content-specific token gets spent, also a global token needs to be spent and the request is only accepted if both tokens have not been spent before. Note that both types of tokens can be generated using the same pre-token.

Policy updates. Policy updates are a means to react to the current or expected network situation. For instance, if for the current policy there is an overload of token-based requests, we can decrease the total number of remaining tokens by adjusting the size of the policy interval. Similarly, if we start with a very small-sized policy, but the webserver could easily handle more requests in the current situation, then we can replace the policy with a larger one. As a change of policy does not render pre-tokens invalid, all current pre-token holders can generate new tokens according to the new policy without interacting with the issuing server. There are different possibilities to adjust the current policy, including:

- Update by *disjunct set*: Independently of the number of tokens a pre-token holder already redeemed for the old policy, it can redeem a number of tokens equal to the size of the new policy.
- Update by *subset*: This is, e.g., obtained when decreasing the upper bound of the interval of a policy. In this case, pre-token holders may redeem a number of tokens with the updated policy which is at most equal to the size of the new policy (as it may have already spent tokens before associated with the new smaller interval). Note that the operator can count the total number of remaining tokens over all pre-token holders which would be available for a subset policy.
- Update by *superset*: This is, e.g., obtained when increasing the upper bound of the policy interval. Again, pre-token holders may redeem at most a number of tokens equal to the size of the new policy and at least equal to the difference of the sizes of new and old policy. Also, one can count how many unspent tokens would be available when doing such an update.

Invalidation of pre-tokens. Updating policies results in new tokens for *all* pre-token holders. If at some point, the number of pre-token holders is considered too large, all pre-tokens can be invalidated at once by changing the key pair (sk_S, pk_S) of the issuing server. Our current

construction does not support expiration dates for pre-tokens to invalidate them automatically. In order to avoid a peak of pre-token issuing requests by users whose pre-tokens have just been invalidated, one can still use the old server public key (in parallel to the new one) for the verification of tokens for a certain transitioning period while the new server secret key is used for issuing new pre-tokens. The policy used for the old tokens should then stay the same or could be cut in size to speed-up the transitioning process. As soon as the holder of an old pre-token redeemed all tokens according to the old policy it will contact the issuing server to the a new pre-token.

Simulating policies with Privacy Pass. In Privacy Pass, we do not have the concept of updatable policies as described above. However, let us consider if and how the effects of policies can be simulated by Privacy Pass.

First, observe that our construction with a fixed non-updatable global policy of size n is essentially functionally equivalent to Privacy Pass issuing a fixed batch of n tokens. In both cases, each user can redeem n tokens for any resource before the issuing server need to be contacted again, in our system to get a fresh pre-token and in Privacy Pass to get a new batch. To realize separate token spending limits per solved CAPTCHA for individual websites, contents, or services, separate issuing (OPRF) keys would be required in parallel. Also, a user would need to explicitly request tokens for a particular resource which is not the case in our system. To approximate the effects of time-dependent policies, i.e., during the current time period only token batches (of size dependent on the period) issued during the same period are valid, one would need to change the issuing key for each time period and adjust the size of issued token batches accordingly. As the change of the issuing key invalidates tokens from the previous period, all users need to first interact with the issuing server again. In our system, old tokens get invalidated by switching to the new policy interval which can then be used by all pre-token holders, where the pre-token could have been issued in the current or any previous period, to generate tokens for the new period.

In general, with Privacy Pass we cannot really implement the main goal of policy updates, i.e., to retrospectively decrease or increase the number of unspent tokens in circulation. All we can do to decrease the number of unspent tokens is to renew the issuing key, which invalidates every unspent token. On the other hand, changing the number of issued tokens by means of a policy update in our system, does not only affect new issuing requests but always also all previous ones. Hence, if we increase the current policy by a larger (disjunct) one, not only new requests result in more tokens but also all old requests. In Privacy Pass, it is possible to only increase the size of new batches without affecting the size of previous ones. In our system, we would need to renew the issuing key to achieve this.

Appendix E. Proof of Theorem 3

Proof: We start with showing *Correctness*. Let $pp \leftarrow \text{Setup}(1^\lambda)$, $(pk_G, sk_G) \leftarrow \text{GroupKG}(pp)$, $x \in \mathcal{X}$,

$((pk_1, pk_2), sk) \leftarrow \text{KG}(pk_G)$ and $(y, (\pi', \tilde{pk}, \tilde{\text{crt}}), \tau) \leftarrow \text{Eval}(pk_G, pk, sk, \text{crt}, x)$. Then, the following holds:

- The public parameters are of the form $pp = (\text{BG}, \mathcal{X}, \mathcal{Y})$, where $\text{BG} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_3, p, g_1, g_2, e)$ defines a bilinear group, the input and output space are defined as $\mathcal{X} = \mathbb{Z}_p$ and $\mathcal{Y} = \mathbb{G}_3$;
- the key pair (pk_G, sk_G) is of the form $pk_G = (pk^{\text{SIG}}, \pi_G)$ and $sk_G = sk^{\text{SIG}}$, such that π_G is a NIZK proof of knowledge for $\text{SIG.VKey}_{\mathcal{R}}(pk^{\text{SIG}}, sk^{\text{SIG}}) = 1$ and $(pk^{\text{SIG}}, sk^{\text{SIG}})$ is a key pair for the EQ- $\mathcal{R}$ signature scheme;
- the secret-key/public-key pair is of the form $sk \in \mathbb{Z}_p$ and $pk = (g_1, g_1^{\alpha})$, i.e., (pk, sk) is a key pair for (an asymmetric version of) the Dodis-Yampolskiy VRF (DY PRF);
- the certificate crt is in the image of $\text{SIG.Sign}_{\mathcal{R}}(sk_G, pk)$, i.e., crt is an EQ- $\mathcal{R}_{\text{DDH}}$ signature for message pk ;
- the output value $y = e(g_1, g_2)^{1/(x+sk)}$ corresponds to the output of the DY PRF;
- the first part of the proof $\pi' = g_2^{\frac{1}{\tau(x+sk)}}$ corresponds to the proof of the DY PRF re-randomized with $1/\tau$ (where τ is the opening information);
- the second part of the proof $(\tilde{pk}, \tilde{\text{crt}})$ corresponds to the public key pk of the DY PRF re-randomized with τ , together with a re-randomized EQ- $\mathcal{R}$ signature $\tilde{\text{crt}}$ of $\tilde{pk} = pk^{\tau}$.

First of all, by the correctness of the EQ- $\mathcal{R}$ signature scheme and completeness of the proof system PS it holds

$$\text{VerGroup}(pk_G) = \text{PS.Ver}(\text{crs}, pk_G, \pi) = 1$$

$$\text{VerCert}(pk_G, pk, \text{crt}) = \text{SIG.Vfy}_{\mathcal{R}}(pk_G, pk, \text{crt}) = 1.$$

By the correctness of the EQ- $\mathcal{R}$ signature scheme for re-randomized signatures it further holds

$$\text{SIG.Vfy}_{\mathcal{R}}(pk_G, \tilde{pk}, \tilde{\text{crt}}) = 1.$$

Further, it holds

$$\begin{aligned} e(\tilde{pk}_1^x \cdot \tilde{pk}_2, \pi') &= e(g_1^{x \cdot \tau} \cdot g_1^{sk \cdot \tau}, g_2^{\frac{1}{\tau(x+sk)}}) \\ &= e(g_1^{\tau(x+sk)}, g_2^{\frac{1}{\tau(x+sk)}}) = e(g_1, g_2) \end{aligned}$$

and

$$e(\tilde{pk}_1, \pi') = e(g_1^{\tau}, g_2^{\frac{1}{\tau(x+sk)}}) = e(g_1, g_2)^{1/(x+sk)} = y$$

due to the bilinearity of e . Hence we have $\text{Ver}(pk_G, x, y, \pi) = 1$. Next, we have

$$\tilde{pk} = pk^{\tau}$$

and hence $\text{Judge}(pk_G, pk, x, y, \tau, \pi) = 1$. Overall, correctness of GVRF follows.

Pseudorandomness. We prove pseudorandomness of GVRF based on the perfect signature adaption of SIG, the zero knowledge and knowledge soundness of PS and the one-more bilinear 2-DDH/DDHI assumption (Definition 17), which is implied by the DDH/DDHI assumption (Lemma 1).

The reduction is shown in Figure 9. First, we can switch the generation of the crs of PS to the trapdoor generation (crs, td) . By the zero knowledge property of

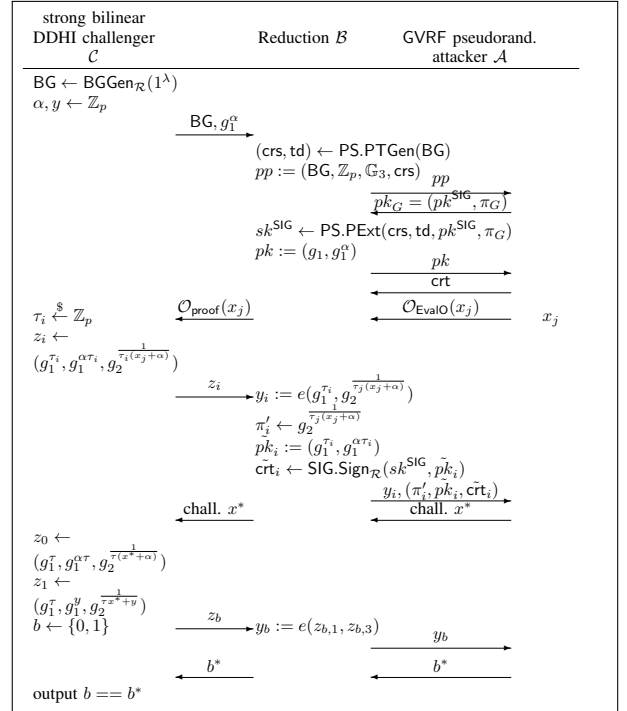


Figure 9: Reduction from the pseudorandomness of GVRF to the one-more bilinear 2-DDH/DDHI assumption (Def. 17). Note that the adversary $\mathcal{A}$ receives access to $\mathcal{O}_{\text{Eval}}(\cdot)$ both before and after sending the challenge x^* , which for simplicity is not depicted in the figure. Further, the reduction $\mathcal{B}$ aborts, if x^* appears at any point as evaluation query, or if $\mathcal{A}$ fails to provide a valid pk_G and/ or valid certificate crt relative to pk .

PS the adversary is able to detect this switch with at most negligible probability.

Next, note that by the knowledge soundness of PS, if the adversary provides a valid proof π_G we can extract a valid secret signing key sk^{SIG} from the adversary except with negligible probability. Therefore, switching the signature generation in the evaluation oracle to fresh signatures does not change the view of the adversary by the perfect signature adaption of SIG.

Further, by setting $sk := \alpha$ we have that the public key and all proofs and output values satisfy the correct distribution, and thus we have that the view of $\mathcal{A}$ is distributed as in the pseudorandomness experiment depicted in Figure 1

Finally, we have that y_b is either the real GVRF value or random, since $y_0 = e(g_1^{\tau}, g_2^{\frac{1}{\tau(x^*+\alpha)}}) = e(g_1, g_2)^{1/(x^*+\alpha)}$ or $y_1 = e(g_1^{\tau}, g_2^{\frac{1}{\tau \cdot x^* + y}})$ (which is distributed uniformly at random as y is random), and hence the success probability of $\mathcal{B}$ is equal to the success probability of $\mathcal{A}$, which concludes the proof.

Unique provability. Let $pp \leftarrow \text{Setup}(1^\lambda)$, $(pk_G, sk_G) \leftarrow \text{GroupKG}(1^\lambda)$. We have to show that for all public keys pk , for all input values x , output values y_0, y_1 , proofs π_0, π_1 and opening information τ_0, τ_1 , for which it holds that $\text{Judge}(pk_G, pk, x, y_0, \pi_0, \tau_0) = \text{Judge}(pk_G, pk, x, y_1, \pi_1, \tau_1) = 1$, it holds $y_0 = y_1$. Let $\pi_b = (\pi'_b, pk_b, \text{crt}_b)$ for $b \in \{0, 1\}$. Then it

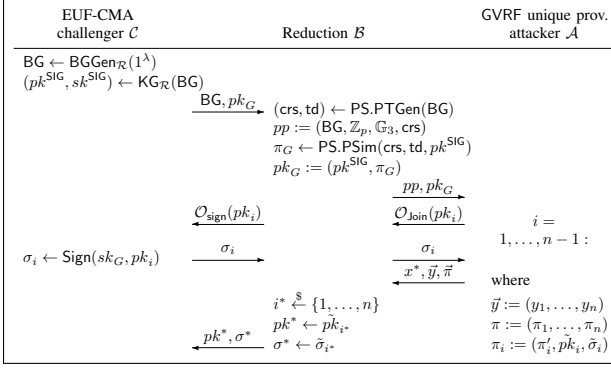


Figure 10: Reduction from the group-bounded provability of GVRF to the EUF-CMA security of SIG.

must hold $\tilde{p}k_0 = pk^{\tau_0}$ and $\tilde{p}k_1 = pk^{\tau_1}$, as well as $\text{Ver}(pk_G, pk, x, y_0, \pi_0) = \text{Ver}(pk_G, pk, x, y_1, \pi_1) = 1$. This implies

- (i) $\tilde{p}k_0 = \tilde{p}k_1^\rho$ for $\rho := \tau_0/\tau_1 \in \mathbb{Z}_p$,
- (ii) $e(\tilde{p}k_{b,1}^* \cdot \tilde{p}k_{b,2}, \pi'_b) = e(g_1, g_2)$ for $b \in \{0, 1\}$
- (iii) $e(\tilde{p}k_{b,1}, \pi'_b) = y_b$ for $b \in \{0, 1\}$.

Together, this yields

$$e(\tilde{p}k_{1,1}^* \cdot \tilde{p}k_{1,2}, \pi'_1) \stackrel{(ii)}{=} e(g_1, g_2) \stackrel{(ii)}{=} e(\tilde{p}k_{0,1}^* \cdot \tilde{p}k_{0,2}, \pi'_0) \\ \stackrel{(i)}{=} e(\tilde{p}k_{1,1}^{x^* \cdot \rho} \cdot \tilde{p}k_{1,2}^\rho, \pi'_0) = e(\tilde{p}k_{1,1}^* \cdot \tilde{p}k_{1,2}, \pi_0^\rho),$$

and hence $\pi'_0 = \pi_1^{1/\rho}$ (iv). With this we obtain

$$y_0 \stackrel{(iii)}{=} e(\tilde{p}k_{0,1}, \pi'_0) \stackrel{(i,iv)}{=} e(\tilde{p}k_{1,1}^\rho, \pi_1^{1/\rho}) = e(\tilde{p}k_{1,1}, \pi'_1) \\ \stackrel{(iii)}{=} y_1$$

as required.

Group-bounded provability. We give a reduction of the group-bounded provability of GVRF to the EUF-CMA security of the signature scheme SIG, using the zero knowledge property of PS which allows the reduction to switch real proofs of correct group key generation to simulated proofs. Recall that an adversary $\mathcal{A}$ breaks unique provability if it produces one more evaluation y of x^* than group members it impersonates.

If the adversary is able generate proof for n distinct image values $y_1, \dots, y_n$, it is able to generate $(pk_1, \tilde{crt}_1), \dots, (pk_n, \tilde{crt}_n)$ such that $\text{SIG.Vfy}_{\mathcal{R}}(pk^{\text{SIG}}, \tilde{p}k_i, \tilde{crt}_i) = 1$, where $(pk_1, \tilde{crt}_1)$ is part of the (valid) proof π_i to preimage y_i . By the proof of unique verifiability, if $y_i \neq y_j$ for all $i \neq j$, we must also have $[\tilde{p}k_i]_{\mathcal{R}_{\text{DDH}}} \neq [\tilde{p}k_j]_{\mathcal{R}_{\text{DDH}}}$ for each $i \neq j$. Since the adversary only received the certificates for $\text{ctr} < n$ public keys from the adversary, we can thus transform an adversary on the group-bounded provability of GVRF to an adversary on the EUF-CMA security on the signature scheme. However, since the reduction cannot recognize equivalence classes either, it needs to guess which of the overall n VRF values supplied by the GVRF attacker constitutes the forgery under pk_G . We present the formal reduction $\mathcal{B}$ in Figure 10.

Since $[\tilde{p}k_i]_{\mathcal{R}_{\text{DDH}}} \neq [\tilde{p}k_j]_{\mathcal{R}_{\text{DDH}}}$ for all $i, j \in \{1, \dots, n\}$, $i \neq j$, the probability that $\mathcal{B}$ picks i^* such that no message of the equivalence class $[\tilde{p}k_{i^*}]$ was ever signed by the

oracle is at least $1/n$. Because reduction $\mathcal{B}$ perfectly implements the unique provability experiment for $\mathcal{A}$, with $\text{Ver}(pk_G, \tilde{p}k_\ell, \tilde{\sigma}_\ell) = 1$ for all $\ell = 1, \dots, n$ it follows that the advantage of reduction $\mathcal{B}$ in winning the EUF-CMA experiment is at least $\Pr[\text{Exp}_{\text{GVRF}, \mathcal{A}}^{\text{gb-prov}}(1^\lambda) = 1]/n$. This concludes the proof.

Unlinkability. We show unlinkability of GVRF under the one-more bilinear 2-DDH/DDHI assumption (omb-2-DDH/DDHI, Def. 17), the perfect signature adaptation of the signature scheme SIG and the zero knowledge and knowledge soundness properties of the proof system PS. Recall that by Lemma 1, the omb-2-DDH/DDHI is implied by the one-more bilinear DDH/DDHI assumption (Lemma 1).

For the proof we first observe that replacing the honest generation of crs by a trapdoor generation is computationally indistinguishable by the zero knowledge property of the proof system. Further, by the knowledge soundness of the proof system, either the reduction aborts (if the adversary fails to provide a public key pk^{SIG} with valid proof π) or the reduction successfully extracts a secret key sk_G with $\text{SIG.VKey}_{\mathcal{R}}(pk_G, sk_G) = 1$, except with negligible probability.

Next, observe that since by assumption our signature scheme satisfies perfect signature adaptation, and since the reduction aborts if the adversary fails to provide valid certificates $\text{crt}_0, \text{crt}_1$ with $\text{SIG.Vfy}_{\mathcal{R}}(pk_G, pk_0, \text{crt}_0) = \text{SIG.Vfy}_{\mathcal{R}}(pk_G, pk_0, \text{crt}_1) = 1$, it does not change the view of the adversary if we replace re-randomized signatures by freshly generated signatures under the secret key sk_G .

Next, we define a set of hybrids $H_0, \dots, H_q$, where q denotes the number of queries to $\mathcal{O}_{\text{Eval}}^b(\cdot)$. Hybrid H_i , for $i = 0, \dots, q$, denotes the unlinkability game (cf. Figure 3) with the modification that the evaluation oracle $\mathcal{O}_{\text{Eval}}^b(\cdot)$ no longer depends on the bit b (and is thus in the following referred to as $\mathcal{O}_{\text{Eval}}(\cdot)$), but instead the first i queries are answered with sk_1 , and queries $i+1, \dots, q$ are answered with sk_0 . Consequently, H_0 denotes the unlinkability game where the adversary has access to oracle $\mathcal{O}_{\text{Eval}}^0(\cdot)$, and H_q is equal to the unlinkability game where the adversary has access to oracle $\mathcal{O}_{\text{Eval}}^1(\cdot)$ (with the only difference that re-randomized signatures are replaced by freshly generated ones, which does not change the view of the adversary as elaborated above).

Now, for each $i = 0, \dots, q-1$, we build an adversary $\mathcal{B}_i$ solving the one-more bilinear 2-DDH/DDHI problem given a PPT adversary $\mathcal{A}$ distinguishing any two of the consecutive hybrids. The reduction is depicted in Figure 11.

For $b^* = 0$, we now have that $\mathcal{B}$ perfectly simulates H_i , since in this case we have:

- $\tilde{p}k_{i+1}^\tau = (z_{j,1}, z_{j,2}) = (g_1^\tau, g_1^{\alpha_0 \tau}) = (g_1, A_0)^\tau = pk_0^\tau$,
- $\pi_{i+1} = z_{j,2} = g_2^{\frac{1}{\tau(\alpha_0 + x_{i+1})}}$ and
- $y_{i+1} = e(z_{j,1}, z_{j,3}) = e(g_1, g_2)^{\frac{1}{\alpha_0 + x_{i+1}}}$,

and thus the $(i+1)$ -st evaluation query is answered using sk_0 (as well as all previous queries, independently of the challenge bit b^*). Similarly, if $b^* = 1$, we obtain that $\mathcal{B}$ perfectly simulates H_{i+1} , since in this case the $(i+1)$ -

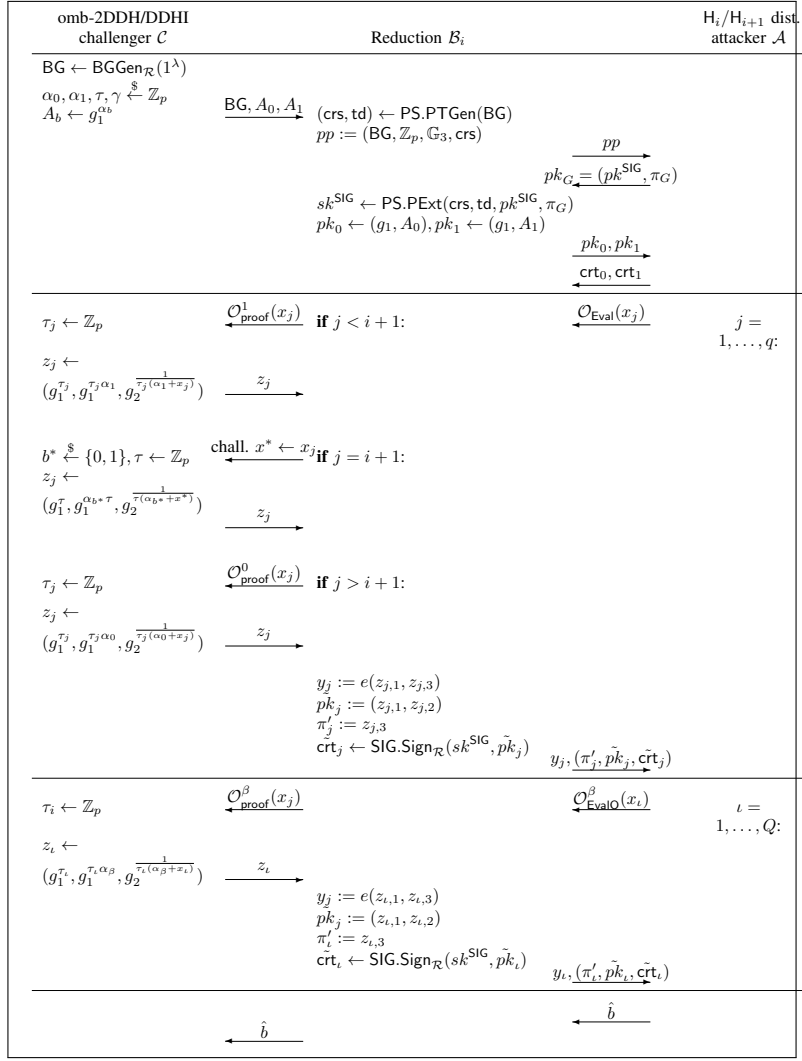


Figure 11: Reduction from the unlinkability of GVRF to the one-more bilinear 2-DDH/DDHI problem. The adversary can query the oracles in arbitrary order. The reduction aborts if the adversary fails to provide a valid public key pk_G or valid certificates crt_0, crt_1 for pk_0 or pk_1 , respectively, or if the adversary asks any x to $\mathcal{O}_{\text{Eval}}(\cdot)$ and $\mathcal{O}_{\text{EvalO}}^\beta(\cdot)$ for any β .

st evaluation query is answered using sk_1 (as well as all following queries, independently of the challenge bit b^*).

Now, let $\mathcal{B}$ denote an adversary which first samples $i \xleftarrow{\$} \{0, \dots, q-1\}$ and then runs $\mathcal{B}_i$ on $\mathcal{A}$. Then, we obtain the following:

$$\begin{aligned}
& \Pr[b^* \leftarrow \mathcal{B}] - \frac{1}{2} \\
&= \frac{1}{2} \Pr[0 \leftarrow \mathcal{B} \mid b^* = 0] + \frac{1}{2} \Pr[1 \leftarrow \mathcal{B} \mid b^* = 1] - \frac{1}{2} \\
&= \frac{1}{2} \Pr[1 \leftarrow \mathcal{B} \mid b^* = 0] - \frac{1}{2} \Pr[1 \leftarrow \mathcal{B} \mid b^* = 1] \\
&= \sum_{j=0}^{q-1} \frac{1}{2q} \cdot (\Pr[1 \leftarrow \mathcal{B}_i \mid b^* = 0 \wedge i = j] \\
&\quad - \Pr[0 \leftarrow \mathcal{B}_i \mid b^* = 1 \wedge i = j]) \\
&= \frac{1}{2q} \cdot \sum_{j=0}^{q-1} (\Pr[1 \leftarrow \mathcal{A} \text{ in } H_i \mid i = j] - \Pr[0 \leftarrow \mathcal{A} \text{ in } H_{i+1} \mid i = j]) \\
&= \frac{1}{2q} \cdot (\Pr[1 \leftarrow \mathcal{A} \text{ in } H_0] - \Pr[1 \leftarrow \mathcal{A} \text{ in } H_q]).
\end{aligned}$$

Since $\Pr[\mathcal{B} \text{ outputs } b^*] - \frac{1}{2}$ is negligible by the one-more bilinear 2-DDH/DDHI assumption, so is the probability that the adversary $\mathcal{A}$ can distinguish between the eval-

uation oracle $\mathcal{O}_{\text{Eval}}^0(\cdot)$ and the evaluation oracle $\mathcal{O}_{\text{Eval}}^1(\cdot)$.

Unique opening. It is left to prove unique opening. Recall that our public keys are of the form $pk = (g_1, A)$. It thus suffices to show that for any fixed input/ output pair (x, y) there cannot exist two public keys pk_0, pk_1 , proofs π_0, π_1 and opening information τ_0, τ_1 such that $\text{Judge}(pk_G, pk_0, x, y, \pi_0, \tau_0) = \text{Judge}(pk_G, pk_1, x, y, \pi_1, \tau_1) = 1$ but $[pk_0]_{\mathcal{R}_{\text{DDH}}} \neq [pk_1]_{\mathcal{R}_{\text{DDH}}}$, since $[pk_0]_{\mathcal{R}_{\text{DDH}}} \neq [pk_1]_{\mathcal{R}_{\text{DDH}}}$ and $pk_{0,1} = pk_{1,1} = g_1$ implies $pk_{0,2} = pk_{2,1}$ as required.

Let $\pi_b = (\pi'_b, \tilde{pk}_b, \tilde{crt}_b)$ the corresponding proof $b \in \{0, 1\}$. Then, the above implies

- (i) $e(\tilde{pk}_{b,1}^x \cdot \tilde{pk}_{b,2}, \pi'_b) = e(g_1, g_2)$ for $b \in \{0, 1\}$
- (ii) $e(\tilde{pk}_{b,1}, \pi'_b) = y$ for $b \in \{0, 1\}$
- (iii) $\tilde{pk}_b = pk_{b^*}^{\tau_b}$ for $b \in \{0, 1\}$.

Further, let $\rho \in \mathbb{Z}_p$ be such that $\tilde{pk}_{0,1} = \tilde{pk}_{1,1}^\rho$ (iv) (note that such a ρ always exists, since $pk_{0,1}, \tilde{pk}_{1,1} \in \mathbb{G}_1$ are

group elements). With this we obtain

$$\begin{aligned} e(\tilde{p}k_{1,1}, \pi'_1) &\stackrel{(i)}{=} y \stackrel{(i)}{=} e(\tilde{p}k_{0,1}, \pi'_0) \stackrel{(iv)}{=} e(\tilde{p}k_{1,1}^\rho, \pi'_0) \\ &= e(\tilde{p}k_{1,1}, \pi'_0{}^{1/\rho}) \end{aligned}$$

and hence $\pi'_0 = \pi'_1{}^{1/\rho}$ (v). With this, we obtain

$$\begin{aligned} e(\tilde{p}k_{1,1}^{x^*} \cdot \tilde{p}k_{1,2}, \pi'_1) &\stackrel{(i)}{=} e(g_1, g_2) \stackrel{(i)}{=} e(\tilde{p}k_{0,1}^{x^*} \cdot \tilde{p}k_{0,2}, \pi'_0) \\ &\stackrel{(iv,v)}{=} e(\tilde{p}k_{1,1}^{x^* \cdot \rho} \cdot \tilde{p}k_{0,2}, \pi'_1{}^{1/\rho}) \end{aligned}$$

which implies

$$e(\tilde{p}k_{1,2}, \pi'_1) = e(\tilde{p}k_{0,2}, \pi'_1{}^{1/\rho})$$

and thus $\tilde{p}k_{0,2} = \tilde{p}k_{1,2}$ and hence $\tilde{p}k_0 = \tilde{p}k_1$, which by (iii) implies $[pk_0]_{\mathcal{R}_{DDH}} = [pk_1]_{\mathcal{R}_{DDH}}$ as required. $\square$

Appendix F. Hardness of Strong Bilinear DDH/DDHI in the Generic Group Model

In the following we show the hardness of our new problem in the generic group model (GGM) [34], [30]. We decided to analyze our new assumption in the GGM instead of the more general algebraic group model (AGM) [21], [32] for the following reasons: Using the AGM, one may only show that a new problem $\mathcal{P}$ can be reduced to another problem $\mathcal{P}'$. Such a reduction is reasonable and serves as evidence for the hardness of $\mathcal{P}$ if $\mathcal{P}'$ is a well-analyzed standard problem, e.g., the q -DL problem. However, in our case, we do not see how we could reduce our problem to a standard problem: As our problem involves rational functions $f(\vec{x}) = \frac{P(\vec{x})}{Q(\vec{x})}$, with P, Q polynomial, in the exponent, we cannot embed a q -DL instance (as one of the components of $\vec{x}$) as it is not clear how to compute the denominator $Q(\vec{x})$. Adding an exponent-inversion oracle to q -DL would allow an embedding but result in a non-standard problem. To alleviate this issue, we could additionally prove the hardness of q -DL with exponent-inversion oracle in the GGM, but we rather decided to show hardness of $\mathcal{P}$ in the GGM in a direct fashion.

Our approach is the following: Instead of studying only the hardness of strong bilinear DDH/DDHI, we consider a very general class of problems, called generalized bilinear decisional problem (GBDP). This leads to re-usable results and highlights the essential properties making a decisional problem hard in the GGM. Note that other general classes of problems have been analyzed in the GGM and AGM. For instance, [9] considers the class of so-called Uber-problems in the GGM where inputs and challenge elements are described by polynomials. [3] analyzes Uber-problems in the AGM and also extends this class of computational problems in several ways, e.g., allowing inputs and challenges to be described by rational functions, adversarial chosen challenges, and decisional oracles. Our GBDP seems closest to a decisional variant of their ‘‘Uber Assumption for Rational Fractions and Flexible Targets’’. However, as [3] restricts to considering computational rather than decisional problems in the AGM, we cannot make use of their results to argue about the hardness of GBDP.

To simplify the hardness proof for (non-trivial) GBDP, we perform two preparative steps: first, we get rid of the bilinear setting and consider an equivalent problem (in the GGM) defined over a single group (called $\mathbb{G}_3$ -projected problem). The intuition behind this simplification is that generic algorithms can only obtain information about the challenge bit by means of (non-trivial) equations between computed group elements, and equations over $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively, can equivalently be checked over the target group by means of the pairing. Second, we get rid of the denominators of the rational functions describing a problem by considering the denominator-free version of the problem where all functions are multiplied by the product of all denominators. As generic algorithms may only learn from equations between elements, this change is oblivious to them.

F.1. The Generic (Bilinear) Group Model

We base our formal description of the GGM for bilinear settings on the generic group model introduced by Shoup [34]. Note that there are alternative formalizations such as the one by Maurer [30]. However, it appears that Shoup’s formalization covers a broader set of algorithms [37].

Let $BG := (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_3, p, g_1, g_2, e)$ be an instance of a bilinear group setting and $\Sigma \subset \{0, 1\}^{\lceil \log_2 p \rceil + 1}$ a set containing p bit strings which allows for an efficient random sampling. In Shoup’s GGM, elements of $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_3$ are represented by random bit strings from Σ and a generic group oracle is responsible for translating encodings to elements and executing the (bilinear) group operations. In the following we will write $[a]_i \in \Sigma$ to denote the encoding of a group element $a \in \mathbb{G}_i$ under some (implicitly defined) random encoding function $[\cdot]_i : \mathbb{G}_i \rightarrow \Sigma$. If $\vec{a} \in G_i^k$ is a vector of elements we define $[\vec{a}]_i := ([a_1]_i, \dots, [a_k]_i)$. Since the groups $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_3$ are all isomorphic to $\mathbb{Z}_p$, we (internally) represent them by the group $(\mathbb{Z}_p, +)$ with generator 1. So, we actually consider $[\cdot]_i : \mathbb{Z}_p \rightarrow \Sigma$.

An algorithm $\mathcal{A}$ in the GGM interacts with a *generic group oracle* $\mathcal{O}$, which computes the group operations and evaluates the pairing on behalf of $\mathcal{A}$. $\mathcal{O}$ receives as input three vectors of group elements (the problem instance)

$$\begin{aligned} I_1 &= (a_{1,1}, \dots, a_{1,n_1}) \in \mathbb{Z}_p^{n_1} (\cong \mathbb{G}_1^{n_1}), \\ I_2 &= (a_{2,1}, \dots, a_{2,n_2}) \in \mathbb{Z}_p^{n_2} (\cong \mathbb{G}_2^{n_2}), \\ I_3 &= (a_{3,1}, \dots, a_{3,n_3}) \in \mathbb{Z}_p^{n_3} (\cong \mathbb{G}_3^{n_3}). \end{aligned}$$

As internal state, $\mathcal{O}$ maintains two types of lists namely element lists L_1, L_2, L_3 , where $L_i \subset \mathbb{Z}_p$, and encoding lists E_1, E_2, E_3 , where $E_i \subset \Sigma$. For an index j let $L_{i,j}$ and $E_{i,j}$ denote the j -th entry of L_i and E_i , respectively. A list E_i contains the encodings of the group elements corresponding to the entries of L_i , i.e., $E_{i,j} = [L_{i,j}]_i$. To determine these encodings, each time an element a should be added to one of the lists L_i , the oracle internally executes the algorithm $\text{Encode}(a, i)$ which returns $E_{i,j}$ if there already exists j with $L_{i,j} = a$ and a fresh encoding $s \xleftarrow{\$} \Sigma \setminus E_i$ otherwise.

Each list L_i is initially populated with the corresponding elements given as part of a problem instance, i.e., $L_i := I_i$. More precisely, an algorithm $\text{Init}(I_1, I_2, I_3)$ is

executed, which for $1 \leq i \leq 3$ and $1 \leq j \leq n_i$ appends $a_{i,j}$ to L_i , computes $[a_{i,j}]_i := \text{Encode}(a_{i,j}, i)$, appends $[a_{i,j}]_i$ to E_i , and also sends the encoding to $\mathcal{A}$.

The algorithm may then query the oracle to perform operations on the elements by providing the corresponding encodings. We may always assume that $\mathcal{A}$ only provides already assigned encodings $[a]_i \in E_i$ as input to the oracle. The following queries are offered by $\mathcal{O}$:

- $\text{GOp}([a]_i, [a']_i, i)$: On input of two encodings $[a]_i, [a']_i$ and a list index $i \in \{1, 2, 3\}$, the oracle first determines the elements $a, a' \in L_i$ by lookup, i.e., it determines indices j and j' with $[a]_i = E_{i,j}$ and $[a']_i = E_{i,j'}$ and returns $L_{i,j}$ and $L_{i,j'}$. Then it computes $c = a + a' \in \mathbb{Z}_p$ and $[c]_i := \text{Encode}(c, i)$, appends c to L_i and $[c]_i$ to E_i , and sends $[c]_i$ to $\mathcal{A}$.
- $\text{BilMap}([a]_1, [a']_2)$: On input of two encodings $[a]_1, [a']_2$, it first determines $a \in L_1$ and $a' \in L_2$ by lookup. Then it computes $c = aa' \in \mathbb{Z}_p$ and $[c]_3 := \text{Encode}(c, 3)$, appends c to L_3 and $[c]_3$ to E_3 , and sends $[c]_3$ to $\mathcal{A}$.

Note that a random group element can be efficiently sampled by $\mathcal{A}$ by using $\text{GOp}(\cdot)$ to raise a generator to some $r \xleftarrow{\$} \mathbb{Z}_p$.

The GGM for a single group can be easily derived from the bilinear version described above. In particular, the generic group oracle only needs to manage one element list $L \subset \mathbb{Z}_p$ and one encoding list $E \subset \Sigma$, and only answers group operation queries $\text{GOp}([a], [a'])$ associated with this group.

F.2. Generalized (Bilinear) Decisional Problem and Relations in the GGM

Instead of only showing the hardness of strong bilinear DDH/DDHI in the GGM, we do this for a broad class of problems which contains our problem as a special instance. We start by defining Generalized Bilinear Decisional Problems.

Definition 19 (Generalized Bilinear Decisional Problem (GBDP)). Let $\mathcal{G}$ be a asymmetric bilinear group generator returning $\text{BG} := (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_3, p, g_1, g_2, e)$. A $(f^{(1)}, f^{(2)}, f^{(3)}, \mathcal{X}, \mathcal{D}, m, \ell)$ -GBDP $\mathcal{P}$ over BG is defined by tuples of rational functions $f^{(1)} = (f_1^{(1)}, \dots, f_{n(1)}^{(1)})$, $f^{(2)} = (f_1^{(2)}, \dots, f_{n(2)}^{(2)})$, $f^{(3)} = (f_1^{(3)}, \dots, f_{n(3)}^{(3)})$, where $f_j^{(1)}, f_j^{(2)}, f_j^{(3)} : \{0, 1\} \times \mathbb{Z}_p^m \rightarrow \mathbb{Z}_p$, $f_1^{(i)} = 1$, a set $\mathcal{X} \subset \mathbb{Z}_p$, a distribution $\mathcal{D}$ on $\mathcal{X}^{m-\ell}$, and the following game played with algorithm $\mathcal{A}$:

```

 $\vec{x} \xleftarrow{\mathcal{D}} \mathcal{X}^{m-\ell}$ 
 $\vec{y} \xleftarrow{\$} \mathbb{Z}_p^\ell$ 
 $b \xleftarrow{\$} \{0, 1\}$ 
 $b' \leftarrow \mathcal{A}(\text{BG}, g_1^{f^{(1)}(b, \vec{x}, \vec{y})}, g_2^{f^{(2)}(b, \vec{x}, \vec{y})}, e(g_1, g_2)^{f^{(3)}(b, \vec{x}, \vec{y})}, \vec{x})$ 
if  $b' = b$  return 1
else return 0

```

We call $\mathcal{P}$ well-defined if the functions $f_j^{(i)}(b, \vec{x}, \vec{y}) = \frac{P_j^{(i)}(b, \vec{x}, \vec{y})}{Q_j^{(i)}(b, \vec{x}, \vec{y})}$ are always well-defined for arbitrary $b \in \{0, 1\}$ and $\vec{x} \in \mathcal{X}$, i.e., $Q_j^{(i)}(b, \vec{x}, \vec{y})$ viewed as a

polynomial in variables $\vec{Y}$ is never zero. In the following we always assume that a GBDP is well-defined without explicitly mentioning this.

Note that in the GGM as defined in Section F.1, the game specified in Definition 19 translates to

```

 $\vec{x} \xleftarrow{\mathcal{D}} \mathcal{X}^{m-\ell}$ 
 $\vec{y} \xleftarrow{\$} \mathbb{Z}_p^\ell$ 
 $b \xleftarrow{\$} \{0, 1\}$ 
 $([f^{(1)}(b, \vec{x}, \vec{y})]_1, [f^{(2)}(b, \vec{x}, \vec{y})]_2, [f^{(3)}(b, \vec{x}, \vec{y})]_3) \leftarrow \mathcal{O}.\text{Init}(f^{(1)}(b, \vec{x}, \vec{y}), f^{(2)}(b, \vec{x}, \vec{y}), f^{(3)}(b, \vec{x}, \vec{y}))$ 
 $b' \leftarrow \mathcal{A}^{\mathcal{O}.\text{GOp}, \mathcal{O}.\text{BilMap}}(p, [f^{(1)}(b, \vec{x}, \vec{y})]_1, [f^{(2)}(b, \vec{x}, \vec{y})]_2, [f^{(3)}(b, \vec{x}, \vec{y})]_3, \vec{x})$ 
if  $b' = b$  return 1
else return 0

```

To simplify our analysis of GBDP problems (over bilinear settings) in the GGM, we will define an appropriate problem over a single group, the former can be reduced to in the GGM. To this end, we first define the class of Generalized Decisional Problems (GDP) analogously to GBDP.

Definition 20 (Generalized Decisional Problem (GDP)).

Let $\mathcal{G}$ be a group generator returning the description $(\mathbb{G}, g, p)$ of a cyclic group of prime order p . The $(\vec{f}, \mathcal{X}, \mathcal{D}, m, \ell)$ -GDP $\mathcal{P}$ over $(\mathbb{G}, g, p)$ is defined by a tuple of rational functions $\vec{f} = (f_1, \dots, f_n)$, where $f_j : \{0, 1\} \times \mathbb{Z}_p^m \rightarrow \mathbb{Z}_p$, a set $\mathcal{X} \subset \mathbb{Z}_p$, a distribution $\mathcal{D}$ on $\mathcal{X}^{m-\ell}$, and the following game played with algorithm $\mathcal{A}$:

```

 $\vec{x} \xleftarrow{\mathcal{D}} \mathcal{X}^{m-\ell}$ 
 $\vec{y} \xleftarrow{\$} \mathbb{Z}_p^\ell$ 
 $b \xleftarrow{\$} \{0, 1\}$ 
 $b' \leftarrow \mathcal{A}((\mathbb{G}, g, p), g^{\vec{f}(b, \vec{x}, \vec{y})}, \vec{x})$ 
if  $b' = b$  return 1
else return 0

```

We call $\mathcal{P}$ well-defined if the functions $f_j(b, \vec{x}, \vec{y}) = \frac{P_j(b, \vec{x}, \vec{y})}{Q_j(b, \vec{x}, \vec{y})}$ are always well-defined for arbitrary $b \in \{0, 1\}$ and $\vec{x} \in \mathcal{X}$, i.e., $Q_j(b, \vec{x}, \vec{y})$ viewed as a polynomial in variables $\vec{Y}$ is never zero. In the following we always assume that a GDP is well-defined without explicitly mentioning this.

The GDP game in the GGM can be described as follows:

```

 $\vec{x} \xleftarrow{\mathcal{D}} \mathcal{X}^{m-\ell}$ 
 $\vec{y} \xleftarrow{\$} \mathbb{Z}_p^\ell$ 
 $b \xleftarrow{\$} \{0, 1\}$ 
 $[f(b, \vec{x}, \vec{y})] \leftarrow \mathcal{O}.\text{Init}(f(b, \vec{x}, \vec{y}))$ 
 $b' \leftarrow \mathcal{A}^{\mathcal{O}.\text{GOp}}(p, [f(b, \vec{x}, \vec{y})], \vec{x})$ 
if  $b' = b$  return 1
else return 0

```

To define an appropriate problem to reduce a GBDP to, we consider the $\mathbb{G}_3$ -projection of a GBDP problem, which essentially asks to determine b (only) given the e -images of all initial inputs over the groups $\mathbb{G}_1$ and $\mathbb{G}_2$. The idea is that algorithms in the GGM may only check equations between group elements which could equivalently be checked over $\mathbb{G}_3$ using the pairing.

Definition 21 ($\mathbb{G}_3$ -Projection). Let $(f^{(1)}, f^{(2)}, f^{(3)}, \mathcal{X}, \mathcal{D}, m, \ell)$ -GBDP be given. Then its $\mathbb{G}_3$ -projection is the $(\vec{f}, \mathcal{X}, m, \ell)$ -GDP defined by $\vec{f} := ((f_i^{(1)} \cdot f_j^{(2)})_{i,j}, f^{(3)})$.

Lemma 2. The $\mathbb{G}_3$ -projection of a well-defined GBDP is always well-defined.

Lemma 3 (Reduction to $\mathbb{G}_3$ -projection). Let $\mathcal{P}$ be a $(f^{(1)}, f^{(2)}, f^{(3)}, \mathcal{X}, \mathcal{D}, m, \ell)$ -GBDP and the $(\vec{f}, \mathcal{X}, \mathcal{D}, m, \ell)$ -GDP $\mathcal{P}'$ be its $\mathbb{G}_3$ -projection. Suppose there exists a generic group algorithm $\mathcal{A}$ solving $\mathcal{P}$ with advantage ϵ using t oracle queries. Then there exists a generic group algorithm $\mathcal{B}$ solving $\mathcal{P}'$ with advantage ϵ using at most $tn_1n_2(2\log(p) + 1)$ oracle queries, where n_i denotes the size of vector $f^{(i)}$.

Proof: The idea is to let $\mathcal{B}$ simulate $\mathcal{O}.\text{GOp}([a]_i, [a']_i, i)$ for $i \in \{1, 2, 3\}$ and $\mathcal{O}.\text{BilMap}([a]_1, [a']_2)$ for $\mathcal{A}$ by applying the group operation and equality tests (via encodings) to the projected elements offered by $\mathcal{B}$'s generic group oracle $\mathcal{O}'$. As pairing queries are for elements for which $\mathcal{B}$ learns a linear representation in the input elements $f_j^{(1)}(b, \vec{x}, \vec{y})$ and $f_{j'}^{(2)}(b, \vec{x}, \vec{y})$, respectively, and (encodings of) the pairings of those input elements are given, those queries can be handled via group operations over the target group:

$$e \left(\prod_j g_1^{z_j f_j^{(1)}(b, \vec{x}, \vec{y})}, \prod_{j'} g_2^{z_{j'}' f_{j'}^{(2)}(b, \vec{x}, \vec{y})} \right) = \prod_j \prod_{j'} e \left(g_1^{f_j^{(1)}(b, \vec{x}, \vec{y})}, g_2^{f_{j'}^{(2)}(b, \vec{x}, \vec{y})} \right)^{z_j z_{j'}'}$$

More precisely, $\mathcal{B}$ manages lists E_1, E_2, E_3 containing encodings for elements of $\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_3$, respectively, just like the original generic group oracle.⁶ The lists $L_1, L_2, L_3 \subset E$ now also contain encodings (sent by $\mathcal{O}'$) representing the projected elements in $\mathbb{G}$ to which $\mathcal{B}$ has access via its own generic group oracle $\mathcal{O}'$. Those lists are used to map operations on L_1, L_2, L_3 to operations on L . We define a modified encoding function $\text{Encode}'([a], i)$ which is executed by $\mathcal{B}$ every time an encoded element should be added to one of the lists during the initialization or query phase. $\text{Encode}'([a], i)$ determines whether there exists some index j such that $L_{i,j} = [a]$. If this is the case, it returns $E_{i,j}$ as $[a]_i$, otherwise, it randomly samples a fresh encoding $s \xleftarrow{\$} \Sigma \setminus E_i$ and return it as $[a]_i$.⁷

During $\mathcal{O}'$.Init, $\mathcal{B}$ receives encodings

$$\left(([f_i^{(1)}(b, \vec{x}, \vec{y}) \cdot f_j^{(2)}(b, \vec{x}, \vec{y})]_{i,j}), ([f_i^{(3)}(b, \vec{x}, \vec{y})]_i) \right)$$

6. Actually, we could use the original encodings sent by $\mathcal{B}$'s oracle to represent elements of $\mathbb{G}_3$, i.e., $E_3 := E$. However, to simplify the description we simulate all groups in the same way.

7. Note that we cannot simply use the encodings $[x]$ received from $\mathcal{O}'$ to represent elements from all three groups but rather have to define a separate encoding function for each group since for $\mathcal{O}'$ all elements are "considered to be from the same group". To put it simply, g_1^x and g_2^x is treated as equal by $\mathcal{O}'$ and receive the same encoding $[x]$. So we need to keep track whether $[x]$ is associated with $\mathbb{G}_1$ or $\mathbb{G}_2$ and check whether we have already seen this encoding for the corresponding group, and assign an old or fresh encoding $[x]_i$ accordingly.

from its oracle. Remind that we assume that $f_1^{(1)}(b, \vec{x}, \vec{y}) = 1$ and $f_1^{(2)}(b, \vec{x}, \vec{y}) = 1$ to always include the default generators in a problem instance. For $1 \leq i \leq n_1$ it appends $[a_i] := [f_i^{(1)}(b, \vec{x}, \vec{y}) \cdot 1]$ to L_1 and $[a_i]_1 := \text{Encode}'([a_i], 1)$ to E_1 , and sends $[a_i]_1$ to $\mathcal{A}$. Similarly, it initializes L_2 and E_2 with $[1 \cdot f_i^{(2)}(b, \vec{x}, \vec{y})]$ and its $[\cdot]_2$ encodings, respectively, for $1 \leq i \leq n_2$. The list L_3 is initialized with $[a_i] := [f_i^{(3)}(b, \vec{x}, \vec{y})]$ and E_3 with $[a_i]_3 := \text{Encode}'([a_i], 3)$ for $1 \leq i \leq n_3$. Additionally, $\mathcal{B}$ stores the encodings $([f_i^{(1)}(b, \vec{x}, \vec{y}) \cdot f_j^{(2)}(b, \vec{x}, \vec{y})]_{i,j})$ for further use when answering pairing queries from $\mathcal{A}$.

Queries from $\mathcal{A}$ are answered by $\mathcal{B}$ in the following way:

- $\text{GOp}([a]_i, [a']_i, i)$: On input of two encodings $[a]_i, [a']_i$ and a list index $i \in \{1, 2, 3\}$, $\mathcal{B}$ determines indices j and j' with $[a]_i = E_{i,j}$ and $[a']_i = E_{i,j'}$. Then queries its oracle $[c] \leftarrow \mathcal{O}'.\text{GOp}(L_{i,j}, L_{i,j'})$, encodes the result $[c]_i := \text{Encode}'([c], i)$, and appends $[c]_i$ to L_i and $[c]_i$ to E_i , and sends $[c]_i$ to $\mathcal{A}$. Note that by observing the group operation, $\mathcal{B}$ knows the (linear) representation of each c in terms of the initially given elements $f_j^{(i)}(b, \vec{x}, \vec{y})$, i.e., it knows z_j s.t. $c = \sum_j z_j f_j^{(i)}(b, \vec{x}, \vec{y})$.
- $\text{BilMap}([a]_1, [a']_2)$: On input of two encodings $[a]_1, [a']_2$, it first determines $[a] \in L_1$ and $[a'] \in L_2$ by lookup. Let z_j and $z_{j'}'$ be the coefficients belonging to the linear representations of a and a' in terms of the initially given elements $f_j^{(1)}(b, \vec{x}, \vec{y})$ and $f_{j'}^{(2)}(b, \vec{x}, \vec{y})$, respectively, which are known to $\mathcal{B}$ (as explained above). Then for $1 \leq j \leq n_1, 1 \leq j' \leq n_2$, $\mathcal{B}$ first computes $[c_{j,j'}] := [z_j z_{j'}' f_j^{(1)}(b, \vec{x}, \vec{y}) f_{j'}^{(2)}(b, \vec{x}, \vec{y})]$ by applying $\mathcal{O}'.\text{GOp}$ to $[f_j^{(1)}(b, \vec{x}, \vec{y}) f_{j'}^{(2)}(b, \vec{x}, \vec{y})]$. Using square-and-multiply this can be done using at most $2n_1n_2 \log(p)$ queries to $\mathcal{O}'$. After that, it computes $[c] := [\sum_{j,j'} c_{j,j'}]$ by applying $\mathcal{O}'.\text{GOp}$ to the $[c_{j,j'}]$. This requires at most n_1n_2 queries. Finally, it appends $[c]$ to L_3 , $[c]_3 := \text{Encode}'([c], 3)$ to E_3 , and sends $[c]_3$ to $\mathcal{A}$.

Note that if $\mathcal{B}$ is given an instance of $\mathcal{P}'$ for $b = 1$ it perfectly simulates an instance of $\mathcal{P}$ for $b = 1$ in the GGM. The same holds for $b = 0$. Thus, if $\mathcal{B}$ simply outputs what $\mathcal{A}$ outputs, it can guess b in the game for $\mathcal{P}'$ with the same probability as $\mathcal{A}$ does in the game for $\mathcal{P}$. In the worst case $\mathcal{A}$ sends t BilMap queries to $\mathcal{B}$ which results in $t(2n_1n_2 \log(p) + n_1n_2)$ queries to $\mathcal{O}'$. $\square$

To simplify the analysis of GDP problems in the GGM, we want to reduce them to problems which do not contain quotients in their description of given group elements but only polynomials. As generic algorithms can only check equalities between group elements and equations still hold if we multiply them with non-zero denominators on both sides, they do not notice any difference. To this end, we define the denominator-free version of a GDP.

Definition 22 (Denom-Free GDP). Let a $(\vec{f}, \mathcal{X}, \mathcal{D}, m, \ell)$ -GDP $\mathcal{P}$ be given and let the polynomials P_j, Q_j denote the nominator and denominator of $f_j = \frac{P_j}{Q_j}$, respectively. If $Q_j = 1$ for all $j \in \{1, \dots, n\}$ we

call $\mathcal{P}$ denom-free. The denom-free version of $\mathcal{P}$ is defined by the denom-free $(\vec{f}', \mathcal{X}, \mathcal{D}, m, \ell)$ -GDP $\mathcal{P}'$ with $f'_j := P_j \prod_{i \neq j} Q_i$.

Lemma 4 (Reduction to Denom-Free GDP). Let $\mathcal{P}$ denote a $(\vec{f}, \mathcal{X}, \mathcal{D}, m, \ell)$ -GDP and $\mathcal{P}'$ be its denom-free version. Suppose there exists a generic group algorithm $\mathcal{A}$ solving $\mathcal{P}$ with advantage ϵ using t oracle queries. Then there exists a generic group algorithm $\mathcal{B}$ which does t queries to solve $\mathcal{P}'$ with success probability at least $\epsilon - \frac{\sum_i \deg(Q_i)}{p}$, where $Q_i = Q_i(b, \vec{x}, \vec{Y})$ are the denominator polynomials of f_i viewed as polynomials in variables $\vec{Y}$ (treating b and $\vec{x}$ as coefficients).

Proof: First note that since $\mathcal{P}$ is well-defined all polynomials $Q_i = Q_i(b, \vec{x}, \vec{Y})$ are non-zero. If one of the polynomials $Q_i = Q_i(b, \vec{x}, \vec{Y})$ if evaluated with random $\vec{y} \xleftarrow{\$} \mathbb{Z}_p^\ell$ becomes zero, then the corresponding input element f_i of $\mathcal{P}$ is not well-defined. However, for $\mathcal{P}'$ the corresponding element f'_i is still well-defined. Thus, in this case a game for $\mathcal{P}$ could differ from a game for $\mathcal{P}'$. But this happens only with probability at most $\frac{\sum_i \deg(Q_i)}{p}$ using the Schwartz-Zippel Lemma [33]. Let us assume this simulation failure event does not happen in the following.

In the reduction, $\mathcal{B}$ just forwards queries and responses between $\mathcal{A}$ and its own generic group oracle, and finally outputs what $\mathcal{A}$ outputs. Note that the only information about the problem instance $\mathcal{A}$ might obtain in the GGM are equalities between encodings. Considering Encode, two computed elements $a = \sum_i z_i f_i(b, \vec{x}, \vec{y})$ and $a' = \sum_i z'_i f_i(b, \vec{x}, \vec{y})$ (represented in terms of the initial input) in a game for $\mathcal{P}$ are assigned the same encoding if $a = a'$ and different encodings if $a \neq a'$. These equations also holds if we would multiply both sides with $\prod_i Q_i(b, \vec{x}, \vec{y})$ which is assumed to be non-zero (see above). Now observe that the resulting equations are exactly the equations that the generic group oracle checks in a game for $\mathcal{P}'$. Thus, $\mathcal{B}$ perfectly simulates a game for an instance of $\mathcal{P}$ unless the simulation failure event defined above happens. $\square$

Finally, to prove hardness of a GDP in the GGM, the problem must not be trivially solvable. A GDP is trivially solvable if there exists an equation that holds for $b = 0$ but not for $b = 1$, or vice versa, for arbitrary choice of the secrets $\vec{y}$.

Definition 23 (Non-Trivial GDP). A $(\vec{f}, \mathcal{X}, \mathcal{D}, m, \ell)$ -GDP is called non-trivial over $(\mathbb{G}, g, p)$ if there are no $z_i \in \mathbb{Z}_p$, $\vec{x} \xleftarrow{\mathcal{D}} \mathcal{X}^{m-\ell}$, $b \in \{0, 1\}$ s.t.

$$\sum_i z_i f_i(b, \vec{x}, \vec{Y}) = 0 \text{ and } \sum_i z_i f_i(1 - b, \vec{x}, \vec{Y}) \neq 0$$

Lemma 5 (Non-Triviality of Denom-Free Version). Let $\mathcal{P}$ be a non-trivial $(\vec{f}, \mathcal{X}, \mathcal{D}, m, \ell)$ -GDP. Then the denom-free version $\mathcal{P}'$ of $\mathcal{P}$ is also non-trivial.

Proof: Multiplying both sides of the equations in Definition 23 for $\mathcal{P}$ with $\prod_{i=1}^n Q_i$ leads to equivalent equations if $\prod_{i=1}^n Q_i \neq 0$. If $\prod_{i=1}^n Q_i = 0$ then $\mathcal{P}$ was already not well-defined. The resulting equations belong to $\mathcal{P}'$. $\square$

We may also define non-triviality of GBDP by considering its $\mathbb{G}_3$ -projection, which is a GDP.

Definition 24 (Non-Trivial GBDP). A GBDP is called non-trivial if its $\mathbb{G}_3$ -projection is non-trivial.

F.3. Generic Hardness of Non-Trivial GBDP

We first prove hardness in the GGM for the broad class of non-trivial denom-free GDP before we derive the hardness of non-trivial GBDP and finally the hardness of strong bilinear DDH/DDHI as a special case.

Theorem 4 (Generic Hardness of Non-Trivial Denom-Free GDP). Let $\mathcal{P}$ be a non-trivial denom-free $(\vec{f}, \mathcal{X}, \mathcal{D}, m, \ell)$ -GDP. Let n be the size of vector $\vec{f}$ and $d := \max_i(\deg(P_i))$, where $P_i = f_i(b, \vec{x}, \vec{Y})$ is the polynomial describing f_i viewed as polynomial in variables $\vec{Y}$ (treating b and $\vec{x}$ as coefficients). Then any generic group algorithm $\mathcal{A}$ doing t oracle queries can solve $\mathcal{P}$ with advantage at most $\frac{2(t+n)^2 d}{p}$.

Proof: We start with the real game for $b = 1$ and modify Encode until we end up with the real game for $b = 0$. Unless a (rare) failure event occurs the modifications are indistinguishable.

Game G_1 . This is the real game for $b = 1$ where $\mathcal{A}$ receives the encodings $[f_i(1, \vec{x}, \vec{y})]$ for $\vec{x} \xleftarrow{\mathcal{D}} \mathcal{X}^{m-\ell}$ and $\vec{y} \xleftarrow{\$} \mathbb{Z}_p^\ell$.

Game G_2 . In this game, we change the way how initially given and computed elements are represented internally by the oracle. We represent all elements which are added to L by formal polynomials in variables $\vec{Y}$, i.e., elements have the form $a = \sum_i z_i f_i(1, \vec{x}, \vec{Y})$. To determine encodings, the polynomials will be evaluated with the $\vec{y} \in \mathbb{Z}_p^\ell$ which have been chosen in the beginning: Encode(a) checks whether there exists j with $L_j(\vec{y}) = a(\vec{y})$ (over $\mathbb{Z}_p$). If this is the case, the encoding E_j is returned, otherwise a fresh encoding $s \xleftarrow{\$} \Sigma \setminus E$ is sampled. As this change is purely conceptual, the advantage of $\mathcal{A}$ in G_1 and G_2 is identical.

Game G_3 . In this game, we modify Encode again to do equality checks over $\mathbb{Z}_p[\vec{Y}]$: Encode(a) checks whether there exists j with $L_j = a$ (over $\mathbb{Z}_p[\vec{Y}]$). If this is the case, the encoding E_j is returned, otherwise a fresh encoding $s \xleftarrow{\$} \Sigma \setminus E$ is sampled. Clearly, this modification may lead to a difference between G_3 and G_2 if for an element a , the encoding function Encode(a) returns an old encoding in G_3 but a fresh encoding in G_2 , or vice versa. The former would happen if there exists an index j with $L_j - a = 0$ over $\mathbb{Z}_p[\vec{Y}]$ but $(L_j - a)(\vec{y}) \neq 0$ over $\mathbb{Z}_p$. This is impossible. The latter would happen if $L_j - a \neq 0$ over $\mathbb{Z}_p[\vec{Y}]$ but $(L_j - a)(\vec{y}) = 0$ over $\mathbb{Z}_p$ for uniformly chosen $\vec{y} \xleftarrow{\$} \mathbb{Z}_p^\ell$. For a fixed L_j and a , this happens with probability at most $\frac{\deg(L_j - a)}{p} \leq \frac{\max_i(\deg(P_i))}{p}$ according to the Schwartz-Zippel Lemma [33]. Since the total number of pairs of such polynomials can be upper bounded by $(t + n)^2$, the probability of such a simulation failure can be upper bounded by $\frac{(t+n)^2 \max_i(\deg(P_i))}{p}$. Note that unless this failure event happens, G_3 and G_2 are identical.

Game G_4 . In this game, we switch the challenge bit b from 1 to 0, i.e., $\mathcal{A}$ initially receives the encodings $[f_i(0, \vec{x}, \vec{Y})]$ for $\vec{x} \xleftarrow{\mathcal{D}} \mathcal{X}^{m-\ell}$. Again, this change might

result in a difference between G_4 and G_3 if for an element a , the encoding function $\text{Encode}(a)$ returns an old encoding in G_4 but a fresh encoding in G_3 , or vice versa. This would happen if $L_j - a = \sum_i z_i f_i(0, \vec{x}, \vec{Y}) = 0$ but $\sum_i z_i f_i(1, \vec{x}, \vec{Y}) \neq 0$, or vice versa. However, this is excluded by the non-triviality property of $\mathcal{P}$.

Game G_5 . This game reverts the change done in G_3 , i.e., to determine encodings, polynomials are evaluated with $\vec{y}$ again: $\text{Encode}(a)$ checks whether there exists j with $L_j(\vec{y}) = a(\vec{y})$ (over $\mathbb{Z}_p$). Similarly, this change can lead to the simulation failure (so a difference between G_5 and G_4) already described in G_3 whose probability can be upper bounded by $\frac{(t+n)^2 \max_i(\deg(P_i))}{p}$.

Game G_6 . This game reverts the conceptual change of G_2 and is equal to the real game for $b = 0$. Elements are not represented by polynomials anymore, i.e., $\mathcal{A}$ receives the encodings $[f_i(0, \vec{x}, \vec{y})]$ for $\vec{x} \xleftarrow{\mathcal{P}} \mathcal{X}^{m-\ell}$ and $\vec{y} \xleftarrow{\mathcal{S}} \mathbb{Z}_p^\ell$. Clearly, G_6 and G_5 are perfectly indistinguishable.

In summary, we have shown that G_1 and G_6 are perfectly indistinguishable unless a failure event occurs in G_3 or G_5 which happens with probability at most $\frac{2(t+n)^2 \max_i(\deg(P_i))}{p}$.

□

To show hardness of a non-trivial GBDP, we can restrict to consider the denom-free version of its $\mathbb{G}_3$ -projection (cf. Lemma 3 and 4). To apply Theorem 4 to the latter, the denom-free version needs to be non-trivial which follows from Lemma 5.

Theorem 5 (Generic Hardness of Non-Trivial GBDP).

Let $\mathcal{P}$ be a non-trivial $(f^{(1)}, f^{(2)}, f^{(T)}, \mathcal{X}, \mathcal{D}, m, \ell)$ -GBDP and let $P_j^{(i)}, Q_j^{(i)}$ denote the numerator and denominator polynomial of $f_j^{(i)}(b, \vec{x}, \vec{Y}) = \frac{P_j^{(i)}(b, \vec{x}, \vec{Y})}{Q_j^{(i)}(b, \vec{x}, \vec{Y})}$

viewed as polynomials in variables $\vec{Y}$ (treating b and $\vec{x}$ as coefficients). Let n_i denote the size of vector $f^{(i)}$ and $d' := \max_{i,j}(\deg(P_j^{(i)}))$, and $d'' := \sum_{i,j} \deg(Q_j^{(i)})$. Then any generic group algorithm $\mathcal{A}$ doing t oracle queries can solve $\mathcal{P}$ with advantage at most $\frac{2(n_1 n_2 (2t \log(p) + t + 1) + n_3)^2 (d' + d'') + d''}{p}$.

Proof: This follows immediately by applying Lemma 3, 4, 5 and Theorem 4. Let $\mathcal{P}'$ denote the $\mathbb{G}_3$ -projection of $\mathcal{P}$ and $\mathcal{P}''$ the denom-free version of $\mathcal{P}'$. Furthermore, let $\mathcal{B}$ and $\mathcal{C}$ denote generic group algorithms for $\mathcal{P}'$ and $\mathcal{P}''$, respectively. Then we get the following advantage

$$\begin{aligned} \text{Adv}_{\mathcal{A}}^{\mathcal{P}} &= \text{Adv}_{\mathcal{B}}^{\mathcal{P}'} \\ &\leq \text{Adv}_{\mathcal{C}}^{\mathcal{P}''} + \frac{\sum_{i,j} \deg(Q_j^{(i)})}{p} \\ &= \text{Adv}_{\mathcal{C}}^{\mathcal{P}''} + \frac{d''}{p} \\ &\leq \frac{2(\text{Time}_{\mathcal{C}}^{\mathcal{P}''} + n_1 n_2 + n_3)^2 (d' + d'')}{p} + \frac{d''}{p} \\ &\leq \frac{2(t n_1 n_2 (2 \log(p) + 1) + n_1 n_2 + n_3)^2 (d' + d'')}{p} + \frac{d''}{p} \\ &= \frac{2(n_1 n_2 (2t \log(p) + t + 1) + n_3)^2 (d' + d'') + d''}{p} \end{aligned}$$

The first equation follows from Lemma 3, the second from Lemma 4, and the fourth from Theorem 4, which can be applied since Lemma 5 holds. Finally, note that $\text{Time}_{\mathcal{C}}^{\mathcal{P}''} = \text{Time}_{\mathcal{B}}^{\mathcal{P}'} \leq \text{Time}_{\mathcal{A}}^{\mathcal{P}} n_1 n_2 (2 \log(p) + 1)$. □

To apply Theorem 5 to the strong bilinear DDH/DDHI problem, we just need to show that it is non-trivial.

According to Definition 24, this means to show that its $\mathbb{G}_3$ -projection is non-trivial.

Lemma 6. The $\mathbb{G}_3$ -projection of the strong bilinear DDH/DDHI problem (Definition 15) is a non-trivial GDP problem.

Proof: We translate our problem to a $(f^{(1)}, f^{(2)}, f^{(T)}, \mathcal{X}, \mathcal{D}, m, \ell)$ -GBDP, where for better readability, we partially use a double-index notation instead of a single index as in the original definition. This gives us:

$$\begin{aligned} &f_1^{(1)}(b, x_1, \dots, x_{\ell_1+1}, Y_1, Y_2, Y_3, Y_{1,1}, \dots, Y_{\ell_1, \ell_2}) = 1 \\ &f_2^{(1)}(b, x_1, \dots, x_{\ell_1+1}, Y_1, Y_2, Y_3, Y_{1,1}, \dots, Y_{\ell_1, \ell_2}) = Y_1 \\ &f_3^{(1)}(b, x_1, \dots, x_{\ell_1+1}, Y_1, Y_2, Y_3, Y_{1,1}, \dots, Y_{\ell_1, \ell_2}) = b \cdot Y_3 + (1-b)Y_1 Y_2 \\ &f_{i,j}^{(1)}(b, x_1, \dots, x_{\ell_1+1}, Y_1, Y_2, Y_3, Y_{1,1}, \dots, Y_{\ell_1, \ell_2}) = Y_{i,j}, \text{ for } 1 \leq i \leq \ell_1, 1 \leq j \leq \ell_2 \\ &f_{\ell_1+i, \ell_2+j}^{(1)}(b, x_1, \dots, x_{\ell_1+1}, Y_1, Y_2, Y_3, Y_{1,1}, \dots, Y_{\ell_1, \ell_2}) = Y_2 Y_{i,j}, \text{ for } 1 \leq i \leq \ell_1, 1 \leq j \leq \ell_2 \\ &f_1^{(2)}(b, x_1, \dots, x_{\ell_1+1}, Y_1, Y_2, Y_3, Y_{1,1}, \dots, Y_{\ell_1, \ell_2}) = 1 \\ &f_2^{(2)}(b, x_1, \dots, x_{\ell_1+1}, Y_1, Y_2, Y_3, Y_{1,1}, \dots, Y_{\ell_1, \ell_2}) = \frac{1}{bY_3 + (1-b)Y_1 Y_2 + Y_1 x_{\ell_1+1}} \\ &f_{i,j}^{(2)}(b, x_1, \dots, x_{\ell_1+1}, Y_1, Y_2, Y_3, Y_{1,1}, \dots, Y_{\ell_1, \ell_2}) = \frac{1}{Y_{i,j}(Y_2 + x_i)}, \text{ for } 1 \leq i \leq \ell_1, 1 \leq j \leq \ell_2 \\ &f_1^{(3)}(b, x_1, \dots, x_{\ell_1+1}, Y_1, Y_2, Y_3, Y_{1,1}, \dots, Y_{\ell_1, \ell_2}) = 1 \end{aligned}$$

Note that $\ell = \ell_1 \ell_2 + 3$ and $m = \ell + \ell_1 + 1$, where $\ell_1 = |\mathcal{X}| - 1$ and $\ell_2 = q$, in our case. Moreover, the distribution $\mathcal{D}$ selects $x_{\ell_1+1} \xleftarrow{\mathcal{S}} \mathcal{X}$ and $x_i \in \mathcal{X} \setminus \{x_{\ell_1+1}\}$. The $\mathbb{G}_3$ -projection is the $(\vec{f}, \mathcal{X}, \mathcal{D}, m, \ell)$ -GDP defined by $\vec{f} := ((f_i^{(1)} \cdot f_j^{(2)})_{i,j}, f^{(3)})$, which is tedious to roll out.

Regarding Definition 23, to show non-triviality we need to consider a term of the form

$$\begin{aligned} &z_1 + z_2 Y_1 \\ &+ z_3 Y_3 + \sum_{i,j} z_{i,j}^{(1)} Y_{i,j} \\ &+ \sum_{i,j} z_{i,j}^{(2)} Y_2 Y_{i,j} + z_4 \frac{1}{Y_3 + Y_1 x_{\ell_1+1}} \\ &+ z_5 \frac{Y_1}{Y_3 + Y_1 x_{\ell_1+1}} + z_6 \frac{Y_3}{Y_3 + Y_1 x_{\ell_1+1}} \\ &+ \sum_{i,j} z_{i,j}^{(3)} \frac{1}{Y_{i,j}(Y_2 + x_i)} + \sum_{i,j} z_{i,j}^{(4)} \frac{Y_1}{Y_{i,j}(Y_2 + x_i)} \\ &+ \sum_{i,j} z_{i,j}^{(5)} \frac{Y_3}{Y_{i,j}(Y_2 + x_i)} + \sum_{i,j} z_{i,j}^{(6)} \frac{Y_{i,j}}{Y_3 + Y_1 x_{\ell_1+1}} \\ &+ \sum_{i,j} z_{i,j}^{(7)} \frac{Y_2 Y_{i,j}}{Y_3 + Y_1 x_{\ell_1+1}} + \sum_i z_i^{(1)} \frac{1}{Y_2 + x_i} \\ &+ \sum_i z_i^{(2)} \frac{Y_2}{Y_2 + x_i} \\ &+ \sum_{i,j} \sum_{(i',j') \neq (i,j)} z_{i,j}^{(i',j')} \frac{Y_{i,j}}{Y_{i',j'}(Y_2 + x_{i'})} \\ &+ \sum_{i,j} \sum_{(i',j') \neq (i,j)} z_{i+j+1, j+\ell_2}^{(i',j')} \frac{Y_2 Y_{i,j}}{Y_{i',j'}(Y_2 + x_{i'})} \end{aligned} \quad (1)$$

$$\begin{aligned}
& z_1 + z_3 \mathbf{Y}_1 \mathbf{Y}_2 + \sum_{i,j} z_{i,j}^{(2)} \mathbf{Y}_2 Y_{i,j} + z_5 \frac{\mathbf{Y}_1}{\mathbf{Y}_1 \mathbf{Y}_2 + \mathbf{Y}_1 x_{\ell_1+1}} + \sum_{i,j} z_{i,j}^{(3)} \frac{1}{Y_{i,j} (Y_2 + x_i)} + \sum_{i,j} z_{i,j}^{(5)} \frac{\mathbf{Y}_1 \mathbf{Y}_2}{Y_{i,j} (Y_2 + x_i)} + \sum_{i,j} z_{i,j}^{(7)} \frac{\mathbf{Y}_2 Y_{i,j}}{\mathbf{Y}_1 \mathbf{Y}_2 + \mathbf{Y}_1 x_{\ell_1+1}} + \sum_i z_i^{(2)} \frac{\mathbf{Y}_2}{Y_2 + x_i} + \sum_{i,j} \sum_{(i',j') \neq (i,j)} z_{i,j}^{(i',j')} \frac{Y_{i,j}}{Y_{i',j'} (Y_2 + x_{i'})} + \sum_{i,j} \sum_{(i',j') \neq (i,j)} z_{i+\ell_1,j+\ell_2}^{(i',j')} \frac{Y_2 Y_{i,j}}{Y_{i',j'} (Y_2 + x_{i'})} \\
& + z_2 Y_1 + \sum_{i,j} z_{i,j}^{(1)} Y_{i,j} + z_4 \frac{1}{\mathbf{Y}_1 \mathbf{Y}_2 + \mathbf{Y}_1 x_{\ell_1+1}} + z_6 \frac{\mathbf{Y}_1 \mathbf{Y}_2}{\mathbf{Y}_1 \mathbf{Y}_2 + \mathbf{Y}_1 x_{\ell_1+1}} + \sum_{i,j} z_{i,j}^{(4)} \frac{\mathbf{Y}_1}{Y_{i,j} (Y_2 + x_i)} + \sum_{i,j} z_{i,j}^{(6)} \frac{Y_{i,j}}{\mathbf{Y}_1 \mathbf{Y}_2 + \mathbf{Y}_1 x_{\ell_1+1}} + \sum_i z_i^{(1)} \frac{1}{Y_2 + x_i}
\end{aligned} \tag{2}$$

First, note that any term involving Y_3 which is equal to 0, remains zero if one replaces Y_3 by Y_1Y_2 . The interesting case is a non-zero term in Y_3 which becomes zero if we replace Y_3 by Y_1Y_2 . In the following, we show that this cannot happen by arguing that any summand containing Y_3 cannot be eliminated when replacing Y_3 with Y_1Y_2 by adding or subtracting other terms.

$z_4 \frac{1}{Y_1 Y_2 + Y_1 x_{\ell_1+1}}$: We observe that all 4 other terms with the same denominator have variables in the numerators. Setting $x_{\ell_1+1} = 1$ or $x_{\ell_1+1} = 0$ does not change the situation. Particularly, besides the 4 terms from before, we do not need to consider additional terms as none of them contains $Y_1 Y_2$ as part of the denominator.

$z_6 \frac{Y_1 Y_2}{Y_1 Y_2 + Y_1 x_{\ell+1}}$: Similar to the case above, none of the fractions with the same denominator share a monomial $Y_1 Y_2$ in the numerator. Let us consider the simplified fraction $z_5 \frac{Y_2}{Y_2 + x_{\ell+1}}$. Again, this term could only be eliminated using $\sum_i z_i^{(2)} \frac{Y_2}{Y_2 + x_i}$ by setting $x_{\ell+1} = x_i$, which is not allowed by the decisional problem. If we set $x_{\ell+1} = 0$, we can simplify the fraction to z_5 which could be eliminated by setting $z_1 = z_5$. However, observe that this also trivially holds for the corresponding term in Y_3 , so this is not a non-zero term in Y_3 in the first place.

a monomial $Y_{i,j}$ in the numerator. Simplifying the fraction by setting $x_{\ell+1} = 0$ does not change this situation as no other terms besides the ones considered before have denominator $\mathbf{Y}_1 \mathbf{Y}_2$. (The term $\sum_{i,j} z_{i,j}^{(7)} \frac{y_2 Y_{i,j}}{\mathbf{Y}_1 \mathbf{Y}_2 + Y_1 x_{\ell+1}}$ is simplified to $\sum_{i,j} z_{i,j}^{(7)} \frac{Y_{i,j}}{\mathbf{Y}_1}$ in this case, which is of no use.)

$\sum_{i,j} z_{i,j}^{(5)} \frac{Y_1 Y_2}{Y_{i,j} (Y_2 + x_i)}$: There are several terms sharing the same denominator, namely

$$\sum_{i,j} z_{i,j}^{(3)} \frac{1}{Y_{i,j} (Y_2 + x_i)}, \quad \sum_{i,j} z_{i,j}^{(4)} \frac{Y_1}{Y_{i,j} (Y_2 + x_i)},$$

$$\sum_{i,j} \sum_{(i',j') \neq (i,j)} z_{i,j}^{(i',j')} \frac{Y_{i,j}}{Y_{i',j'} (Y_2 + x_{i'})}, \quad \text{and}$$

$$\sum_{i,j} \sum_{(i',j') \neq (i,j)} z_{i,j}^{(i',j')} \frac{Y_2 Y_{i,j}}{Y_{i',j'} (Y_2 + x_{i'})}$$
 but none of them share the numerator $Y_1 Y_2$. Simplifying the considered fractions by setting $x_i = 0$ leads to $\sum_{i,j} z_{i,j}^{(5)} \frac{Y_1}{Y_{i,j}}$. The only terms sharing the same denominator, by setting $x'_i = 0$, is

$$\sum_{i,j} \sum_{(i',j') \neq (i,j)} z_{i,j}^{(i',j')} \frac{Y_{i,j}}{Y_{i',j'}}.$$
 But these terms do not contain the monomial Y_1 as numerator.

Theorem 1 (Hardness of Strong Bilinear DDH/DDHI in GGM). Let $\mathcal{P}$ be the strong bilinear DDH/DDHI problem from Definition 15. Then any generic group algorithm sending at most t queries to the generic group oracle can solve $\mathcal{P}$ with advantage at most $\epsilon \leq (2560(|\mathcal{X}|q + 1)^5(t \log(p) + t + 1)^2)/p$.

$$\begin{aligned}
e &\leq \frac{2(n_1 n_2 (2t \log(p) + t + 1) + n_3)^2 (d' + d'') + d''}{p} \\
&= \frac{2((2|\mathcal{X}|q+3)(|\mathcal{X}|q+2)(2t \log(p) + t + 1) + 1)^2 (2+2(|\mathcal{X}|q+1)) + 2(|\mathcal{X}|q+1)}{p} \\
&\leq \frac{2((4|\mathcal{X}|q+4)(2|\mathcal{X}|q+2)(2t \log(p) + t + 2))^2 (5|\mathcal{X}|q+5)}{p} \\
&\leq \frac{2(8(|\mathcal{X}|q+1))^2 (2t \log(p) + t + 2))^2 (5(|\mathcal{X}|q+1))}{p} \\
&\leq \frac{640(|\mathcal{X}|q+1)^5 (2t \log(p) + 2t + 2)^2}{p} \\
&\leq \frac{2560(|\mathcal{X}|q+1)^5 (2t \log(p) + t + 1)^2}{p}
\end{aligned}$$

Appendix G.

Proof of Lemma 1

Let $\mathcal{A}$ be an adversary on the one-more bilinear 2-DDH/DDHI assumption. Then, we construct an adversary $\mathcal{B}$ on the one-more bilinear DDH/DDHI assumption as follows. On input BG, g_1^α , the adversary chooses $b \xleftarrow{\$} \{0, 1\}$ and $\alpha_b \leftarrow \mathbb{Z}_p$ at random, implicitly sets $\alpha_{1-b} := \alpha$ (without knowing the exponent α), and sends $(\text{BG}, g_1^{\alpha_0}, g_1^{\alpha_1})$ to $\mathcal{A}$. On input of a proof-query $\mathcal{O}_{\text{proof}}^\beta(x)$ by $\mathcal{A}$, it proceeds

as follows: If $\beta = b$ it simulates $\mathcal{O}_{\text{proof}}^\beta$ using α_b . If $\beta = 1 - b$, it forwards x to its own challenge query and forwards the response $(g_1^{\tau_i}, g_1^{\alpha\tau_i}, g_2^{\frac{1}{\tau_i(\alpha+x)}})$ to $\mathcal{A}$. Next, it forwards x^* by $\mathcal{A}$ to its own experiment, and hands the challenge query z^* to $\mathcal{A}$. If x^* appears as query to either of the proof oracles at any point, $\mathcal{B}$ aborts. Otherwise, on output $b' = b$ by $\mathcal{A}$, the adversary $\mathcal{B}$ outputs 0 (“real”) to its own experiment. Else, it outputs 1 (“random”) to its own experiment. Towards the analysis of the success probability of $\mathcal{B}$, note that if $b^* = 0$ (where b^* is the bit chosen by its own experiment), then $\mathcal{B}$ perfectly simulates the one-more bilinear 2-DDH/DDHI game for $\mathcal{A}$ relative to the chosen bit b . Further, if $\mathcal{B}$ was successful (i.e., guesses $b' = b$), then so is $\mathcal{A}$. If $b^* = 1$ on the other hand, the view of $\mathcal{A}$ is independent of the challenge bit b . The probability it outputs 1 to its own experiment is thus exactly $\frac{1}{2}$. Overall, we obtain that $\mathcal{B}$ has advantage

$$\begin{aligned}\epsilon_{\mathcal{B}} &:= \Pr[\mathcal{B} \text{ is successful}] - \frac{1}{2} \\ &= \frac{1}{2} \underbrace{\Pr[\mathcal{B} \text{ is successful} | b^* = 0]}_{=\epsilon_{\mathcal{A}} + \frac{1}{2}} + \frac{1}{2} \underbrace{\Pr[\mathcal{B} \text{ is successful} | b^* = 1]}_{=\frac{1}{2}} \\ &\quad - \frac{1}{2} = \frac{1}{2} \cdot \epsilon_{\mathcal{A}},\end{aligned}$$

where $\epsilon_{\mathcal{A}} := \Pr[\mathcal{A} \text{ is successful}] - \frac{1}{2}$. This concludes the proof.